

AfekAdvisor

A Smart Recommendation System for Computer Science Students

Afeka College of Engineering – Final Project Book

Name: Hila Hindi | **ID:** 211337233

Name: Yuval Ervili | **ID:** 214630089

Program: Computer Science

Project Advisor: Victor Taubkin

Table of Contents

1.	Executive Summary.....	3
2.	Background.....	3
2.1	Problem Description.....	3
2.2	System Users and Existing Challenges.....	4
2.3	Market Survey: Existing Solutions in the Market.....	4
2.4	Solutions Comparison and Justification for the Recommendation System...	5
3.	Objectives.....	6
3.1	Project Goals.....	6
3.2	Objectives and Success Metrics.....	6
4.	Planning.....	7
4.1	Functionality and System Requirements.....	7
4.2	Class Diagram and Entity Structure.....	8
4.3	Logical Data Model (Logical ERD).....	9
4.4	Use Cases.....	11
4.5	Sequence Diagrams.....	14
4.6	Technology Stack and Tools.....	19
4.7	Architectural Patterns and Design Methods.....	20
4.8	Development Methodology.....	22
5.	System Structure & Architecture.....	23
5.1	High-Level System Overview.....	23
5.2	API Interface Specification.....	24
5.3	Server-Side and Client-Side	25
5.4	Graphical Interface.....	26
5.5	Algorithmics and System Logic.....	37
6.	Execution and Implementation.....	43
6.1	Development and Runtime Environment.....	43
6.2	Software Testing.....	44
7.	Comparing Planning vs. Execution.....	46
7.1	Responsiveness, Adherence to Planning, and Changes.....	46
7.2	Difficulties Encountered During Development.....	47
7.3	Discussion and Lessons Learned.....	47
8.	Future Development.....	48
8.1	Project Summary and Conclusions.....	48
8.2	Recommended Future Development.....	49
9.	Bibliography.....	50
10.	Appendices.....	51

1. Executive Summary

In this project, we present AfekAdvisor, a smart system for academic and career guidance for Computer Science students at Afeka. Many students experience a sense of confusion throughout their studies, discovering at advanced stages that their academic choices are not aligned with their aspirations or market demands.

The system bridges this gap using advanced recommendation algorithms that integrate text-based similarity (TF-IDF) and real-time job market data. The software development was executed using advanced technologies (such as React and FastAPI), delivering high-quality results and a personalized, data-driven planning tool tailored to Afeka CS students. By utilizing data analysis and TF-IDF-based text similarity, AfekAdvisor provides the student with a comprehensive, multi-semester overview (Roadmap) of their degree, replacing the reliance on partial or unofficial information with an informed planning tool.

The system's interface is user-friendly, intuitive, and requires no prior learning. All academic data and user preferences are stored in a secure database, allowing for continuous and dynamic tracking of the student's progress throughout the degree. The system serves as an innovative solution, enabling students to enjoy a personalized, career-focused, and data-driven academic planning experience.

2. Background

2.1 Problem Description

During a Computer Science degree at Afeka College, students are required to choose elective courses and seminars that shape their specialization track. This choice directly affects the academic workload and the degree of preparation for the job market.

In practice, the selection process is often based on incomplete information - recommendations from friends, overly general syllabi, or purely technical constraints that fit the course schedule.

Currently, there is no centralized tool that allows for the personalization of the study track based on areas of interest, capabilities, and professional relevance to industry demands. As a result, many students choose tracks that do not align with their aspirations, discovering this only at an advanced and overly late stage of their degree.

2.2 System Users and Existing Challenges

The system is designed for several types of potential users, dealing with various problems in the course selection process:

- **Students at the beginning of their degree (Years 1-2):**
Students who are not yet familiar with the existing specialization tracks in Computer Science and the characteristics of the courses.
- **Advanced students (Years 3-4):**
Students with academic experience who are required to choose advanced courses while weighing workload, interest, prerequisites, and professional preparation.
- **Part-time / Full-time working students:**
Students with significant time constraints, for whom characteristics like mandatory attendance and workload are critical.
- **Students debating between multiple specialization tracks:**
Students looking for a clear comparison between different tracks to choose the right direction for them.

2.3 Market Survey: Existing Solutions in the Market

To address the problem, we conducted a market survey that raised several possible solutions:

1. **Reinforcing the academic counseling system:** Adding counselors and expanding reception hours. This provides personal guidance but depends on personnel availability and entails high operational costs.
2. **Displaying static information:** Centralizing data in a uniform file/website. This solution is simple to implement and requires no complex development, but it is not personalized for the student, and its impact on decision-making is limited.
3. **Rule-based system:** Building a track according to fixed rules (prerequisites, workload). In this solution, the system restricts the student to rigid rules that do not necessarily suit their personal profile or their target role.
4. **Smart Data-Driven Recommendation System (The Proposed Solution):**
A system that integrates a personal profile, academic requirements, and job market data to generate a long-term, personalized study track. This solution is designed to provide optimal personalization alongside high professional relevance.

To examine the professional gap, we reviewed studies on academic recommendation systems. The review revealed that:

- A literature review by Ovtšarenko (2023) reviewed methods for generating personalized e-learning paths, but a gap was found: most recommend individual courses without long-term track planning. This led us to the realization that our system must offer a full Roadmap and not just a single course.
- A paper by Atalla et al. (2023) presented an automated consulting system based on rules and grades, but it ignores personal areas of interest and the market. Our relevance lies in integrating industry demands to make the recommendation more professional.
- A study by Ajanovski (2019) demonstrated the importance of mapping knowledge domains in Computer Science. This provided us with further inspiration for building our professional Roadmap.

2.4 Solutions Comparison and Justification for the Recommendation System

# Solution	Personalization	Significant impact	Cost	Time to implement
1	Medium	Yes	Medium	quick
2	Low	No	Low	quick
3	Medium	Yes	Medium	medium
4	High	Yes	High	long

The selected solution is a smart recommendation system (Solution 4). Although it involves more complex development, it provides the highest value to the student: a combination of academic data, personal areas of interest, and real-time changing job market demands. The system meets the student's need for short response time and personalized guidance for a study track that supports a future career.

3. Objectives

3.1 Project Goals

The overarching goal of our project is to transform the course selection experience for Computer Science students through a personalized technological solution.

Within this goal, we defined the following guiding principles:

1. **Improving the decision-making process:** Transforming the choice from random and based on partial information to an informed and planned choice.
2. **Reducing uncertainty:** Minimizing errors in choosing incorrect tracks or those unsuitable for the student's skills.
3. **Personalization:** Building a study track tailored to the student's needs, capabilities, and aspirations.
4. **Industry connection:** Creating a direct link between the academic course content and current demands in the job market.

3.2 Objectives and Success Metrics

To evaluate the extent to which the project outcomes were met, the following metrics were defined:

- **Response time:** recommendations returned in under 40 seconds (with caching)
- **Functional completeness:** onboarding, 6-semester roadmap, top-10 recommendations
- **Correctness:** prerequisite validation; courses with grade ≥ 60 excluded from recommendations
- **Security:** protected endpoints require valid JWT, bcrypt password hashing

4. Planning

4.1 Functionality and System Requirements

AfekAdvisor was designed as a data-driven Web application, assisting the student in planning their study track (elective courses and seminar) based on a personal profile, academic background, and employment market data

Functional Requirements:

- **User and Profile Management:** A multi-step Onboarding process, user authentication, and preference definition (specialization track, target role).
- **Course Catalog and Classification:** A Course Explorer that includes filtering by tracks, workload, course type, and attendance requirements, alongside the display of full course details.
- **Recommendation Engine:** Generating up to 10 personalized recommendations based on weighting track suitability, job market suitability, and student ratings, while handling prerequisites.
- **Track Planning (Roadmap):** Building a study plan for 6 semesters while considering prerequisites.
- **Employment Market Analysis:** Weekly automatic synchronization from an external source (Adzuna API) to extract relevant skills.

Non-Functional Requirements:

- **Performance:** Displaying a recommended study track in less than 40 seconds.
- **Modularity:** A separated Client-Server architecture (FastAPI on the server-side, React on the client-side).
- **Security:** Password encryption using bcrypt and JWT-based authentication for personal endpoints (profile, recommendations, roadmap, history).
- **Maintenance:** A Docker-based runtime environment for container management.

4.2 Class Diagram and Entity Structure

This diagram presents the object-oriented architecture of the system, separating user data, market data, and academic logic, as well as the logical relationships between them:

- **Student:** Manages identity and academic history (completed courses, reviews, planned courses).
- **StudentProfile:** Stores academic preferences and is linked to Track and JobRole (target role).
- **Course:** A class containing the study content, with a self-relation (has_prerequisite) to build the dependency tree (DAG).
- **JobMarket:** Provides skills data from industry. The recommendation engine uses this data to rank the courses (logical connection).
- **Track and JobRole:** Supporting entities that enable mapping between the student's preferences and market demands.

The full data model is detailed in Figure 4.3.2 (ERD).

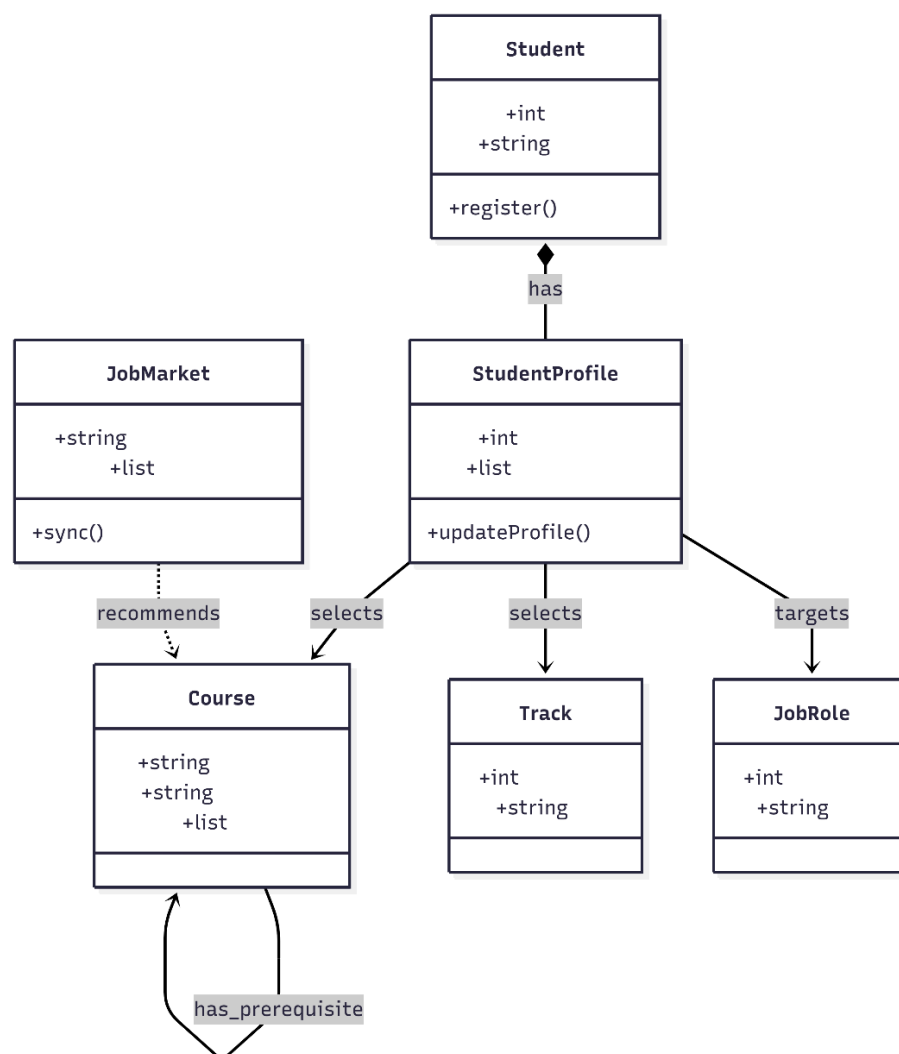


Figure 4.2.1 - Class Diagram

4.3 Logical Data Model (Logical ERD)

Diagram 4.3.2 presents the logical database structure of AfekAdvisor.

The main entities are students (users), student_profiles (academic profile after Onboarding), courses, tracks (study tracks), jobroles (industry target roles), and skills (capabilities).

Many-to-many (M: N) relationships between the profile and tracks/roles, and between courses and skills, are implemented in PostgreSQL using junction tables (not explicitly shown in the simplified diagram).

The tables student_course_history, course_reviews, and planned_courses link students to courses - grade history, reviews, and future planning respectively.

A self-relation on courses (labeled prerequisites) represents the prerequisite dependencies and serves as the basis for building the DAG in the study track engine (Roadmap).

The industry_jobs entity is saved without a foreign key (FK) to other entities; the data is fed from an external API (Adzuna) and serves the course recommendation engine exclusively, via similarity calculation (TF-IDF) rather than a direct ERD connection.

משמעות	שם	סימון
One-to-One	1 : 1	--
One-to-Many	1 : N	--o{
Zero or One	0..1	--o
Many-to-Many	M : N	}o--o{
Primary Key - Unique Identifier	Primary Key	PK
Foreign Key - Link to another table	Foreign Key	FK

Figure 4.3.1 - Relationship Legend

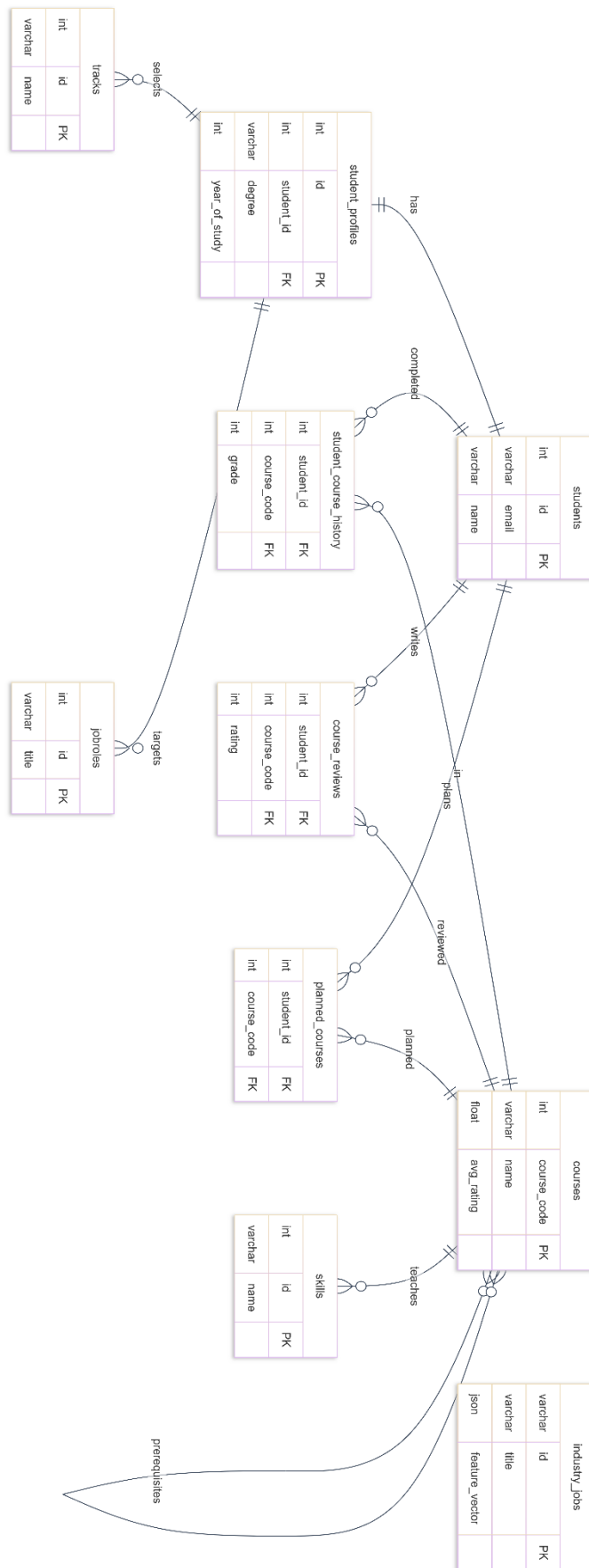


Figure 4.3.2 - ERD of the Database

4.4 Use Cases

Use cases constitute a sequential and verbal analysis of the core functionality in the system, demonstrating the interaction between the student (the main actor) and the system.

Each scenario describes the normal flow to achieve the user's goal, alongside alternative scenarios demonstrating how the system reacts in case of an error or failure to achieve the target.

Two actors are defined in the system:

1. **The student**, who performs actions such as registration and login, profile setup, viewing the roadmap, filtering courses, receiving recommendations, and viewing the Dashboard.
2. **Adzuna API**, an external actor providing employment market data for calculating recommendations.

Data processing, business logic, and information storage are executed within the system boundaries by the Backend server (FastAPI) and the database (PostgreSQL), which are defined as internal components rather than actors.

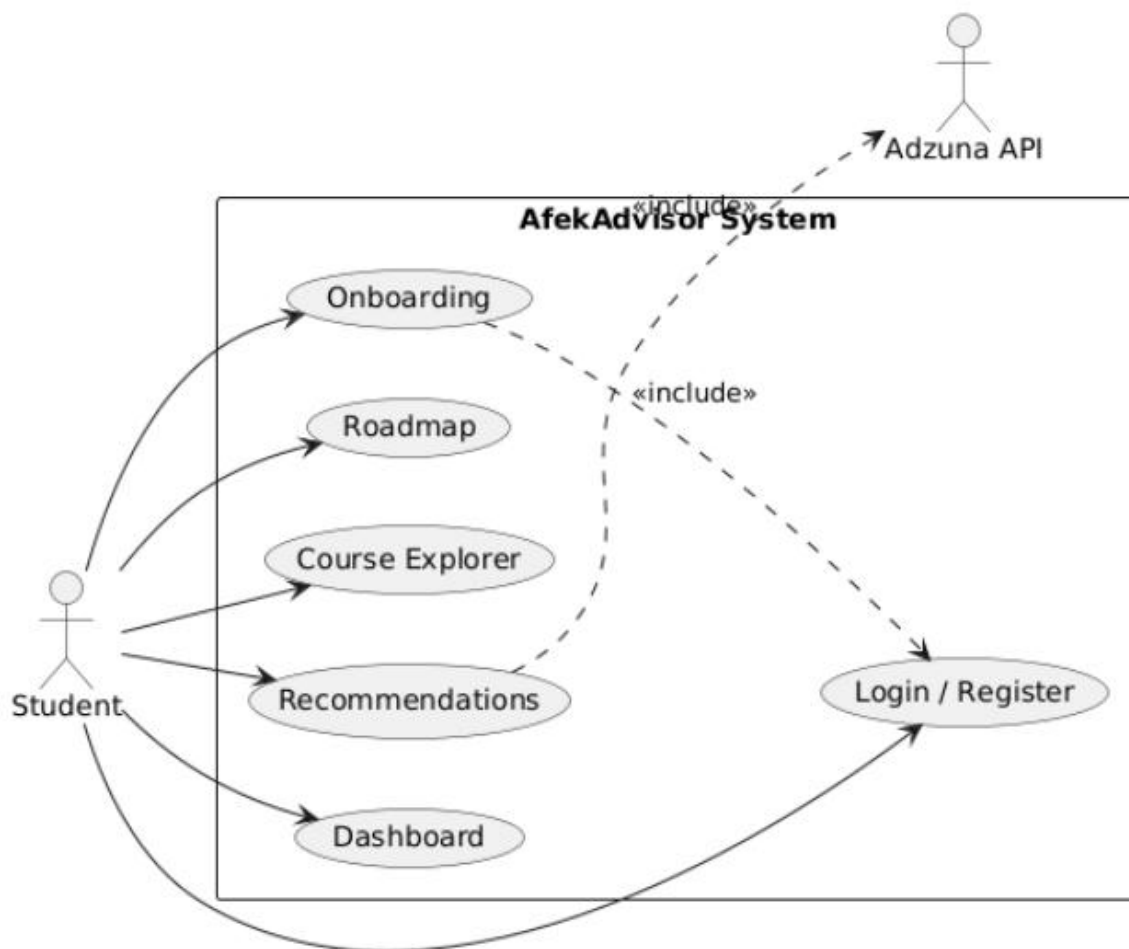


Figure 4.4.1 – Use Case Diagram

1) Student Profile Setup (Onboarding)

1. The user performs the registration and login process and is authenticated by the system.
2. The student goes through 4 structured steps: selecting the degree, academic year, specialization Track, and target JobRole, followed by entering their completed courses list (course code + grade).
3. The system saves the data in the database and flags `onboarding_completed = true`.

Alternative Flow 1 :

- The student attempts to bypass the setup process.
- The system performs an automatic redirect to the Onboarding screen.

Alternative Flow 2:

- Missing data in the course history step.
- The system disables the finish button and prompts: "A grade must be entered for all courses in the list, or unnecessary courses must be deleted".

Alternative Flow 3:

- In steps 1-3, the "Continue" button is disabled until all required fields are filled.

2) Academic Study Planning (Roadmap)

1. The student accesses the "My Track" screen from the navigation menu.
2. The system sends a request to `GET /recommendations/roadmap` and executes a topological sort (Kahn) on the prerequisites DAG.
3. The 6-semester study plan is displayed on the "My Academic Journey" screen - mandatory courses, electives, and seminars, with a status for each course (completed / recommended / future).

Alternative Flow 1:

- The user is missing a prerequisite course.
- The server detects this and automatically injects the missing course into an earlier semester.
- A "Prerequisites" tag will be displayed on the recommendation screen.

Alternative Flow 2:

- Exceptional academic workload in a semester.
- The server automatically redistributes elective courses across semesters (up to 8 courses per semester).

Alternative Flow 3:

- Loading failure.
- An error message is displayed with a "Try Again" button.

3) **Course Filtering in the Course Explorer**

1. The student accesses the "Course Search" interface and defines filters: track, low workload, non-mandatory attendance, and courses whose prerequisites have been met.
2. The system scans the catalog and displays only the courses matching the filters.

Alternative Flow 1:

- No courses match the filter.
- The system displays: "No courses found matching your filter".

4) **Receiving Course Recommendations**

1. The student enters the "Smart Recommendations" screen.
2. The server calculates a weighted score: 50% track suitability, 35% market suitability, 15% review rating - and refreshes market data from Adzuna.
3. The system displays up to 10 elective courses with explanations, required skills, and prerequisites status.

Alternative Flow 1:

- No courses are available.
- The system displays: "No recommendations found at the moment".

Alternative Flow 2:

- Communication error.
- The system displays: "We couldn't load recommendations".

5) **Viewing Dashboard Data**

1. The student accesses the Dashboard to view the academic summary.
2. The system retrieves data from the profile, the top recommendation (#1), and the list of planned courses.
3. Progress data is displayed to the user.

Alternative Flow 1:

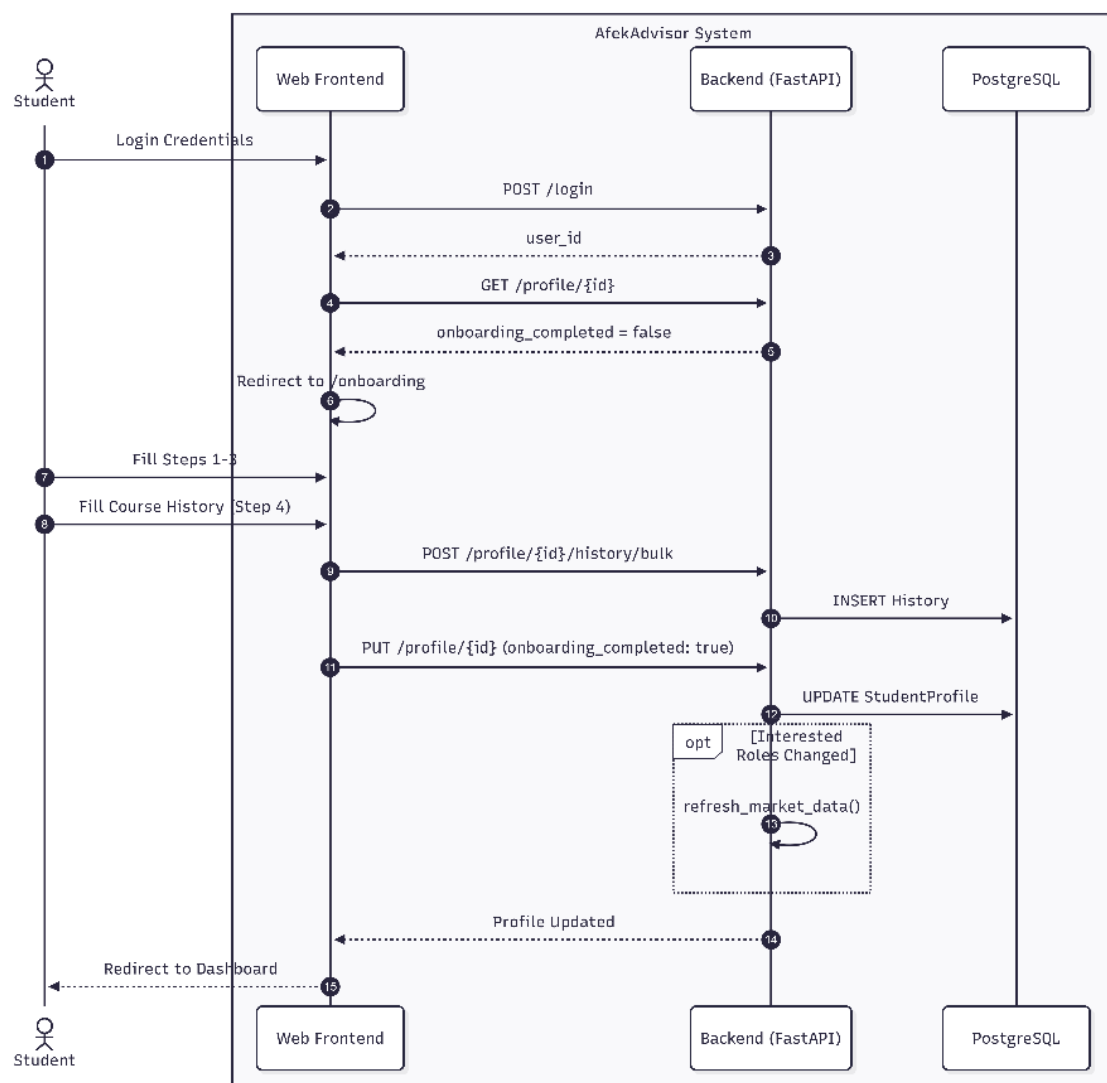
- Data loading failure.
- The system displays a loading indicator (Spinner) until the issue is resolved.

4.5 Sequence Diagrams

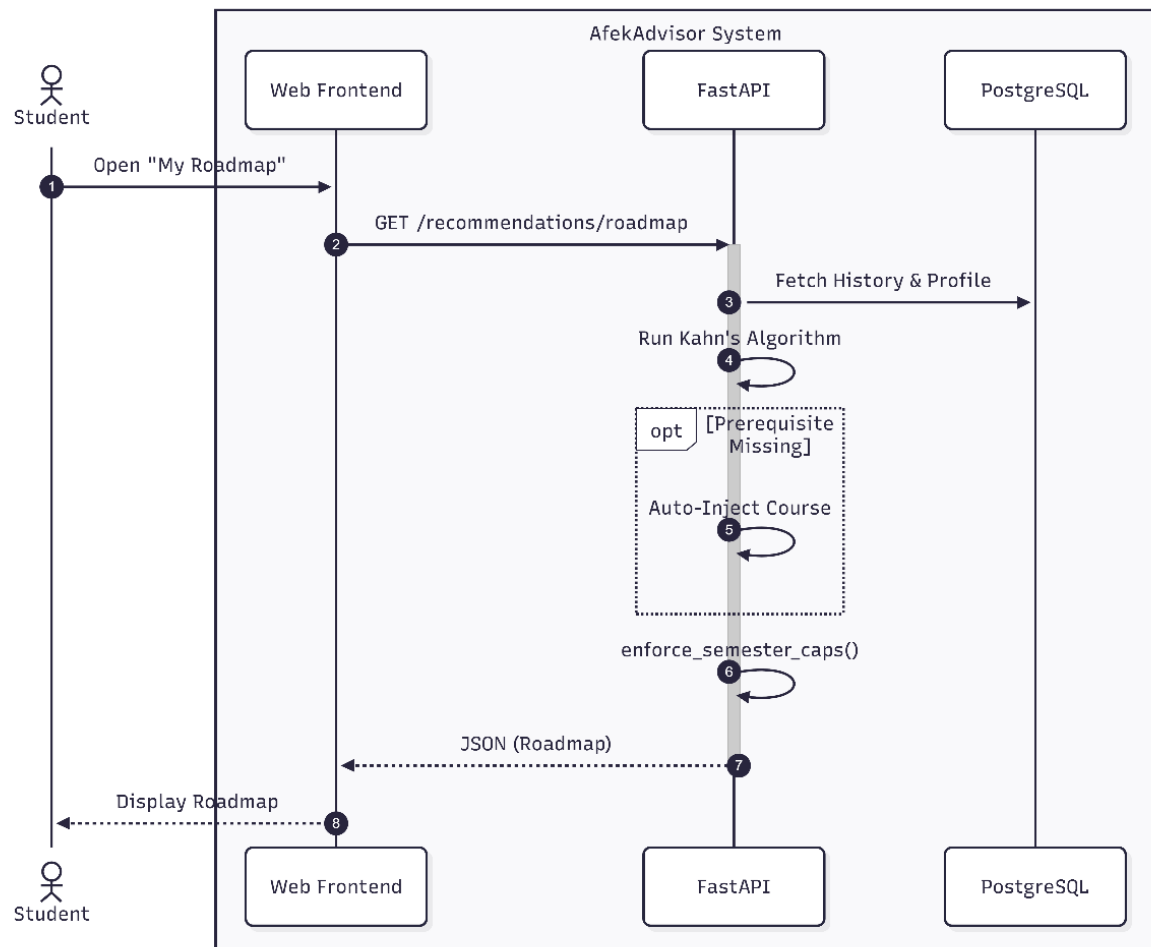
The following diagrams illustrate the dynamics within the system:

- **Receiving Recommendations:** The calculation process including Cache checking and vector similarity calculation.
- **Building a Roadmap:** Topological sorting of courses ensuring compliance with prerequisites.
- **Market Synchronization:** The Pipeline of collecting and processing job postings.

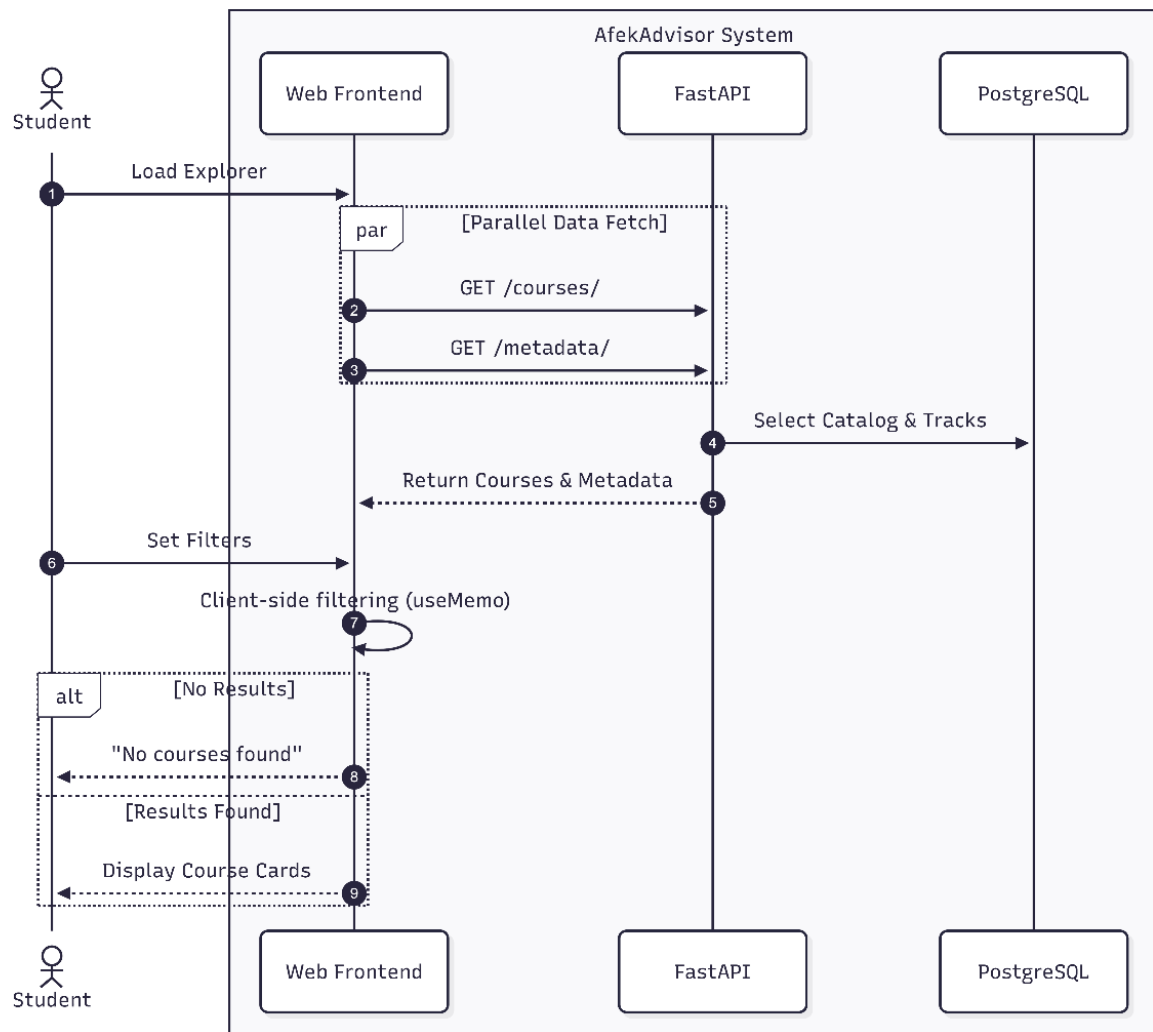
1) Student Profile Setup (Onboarding)



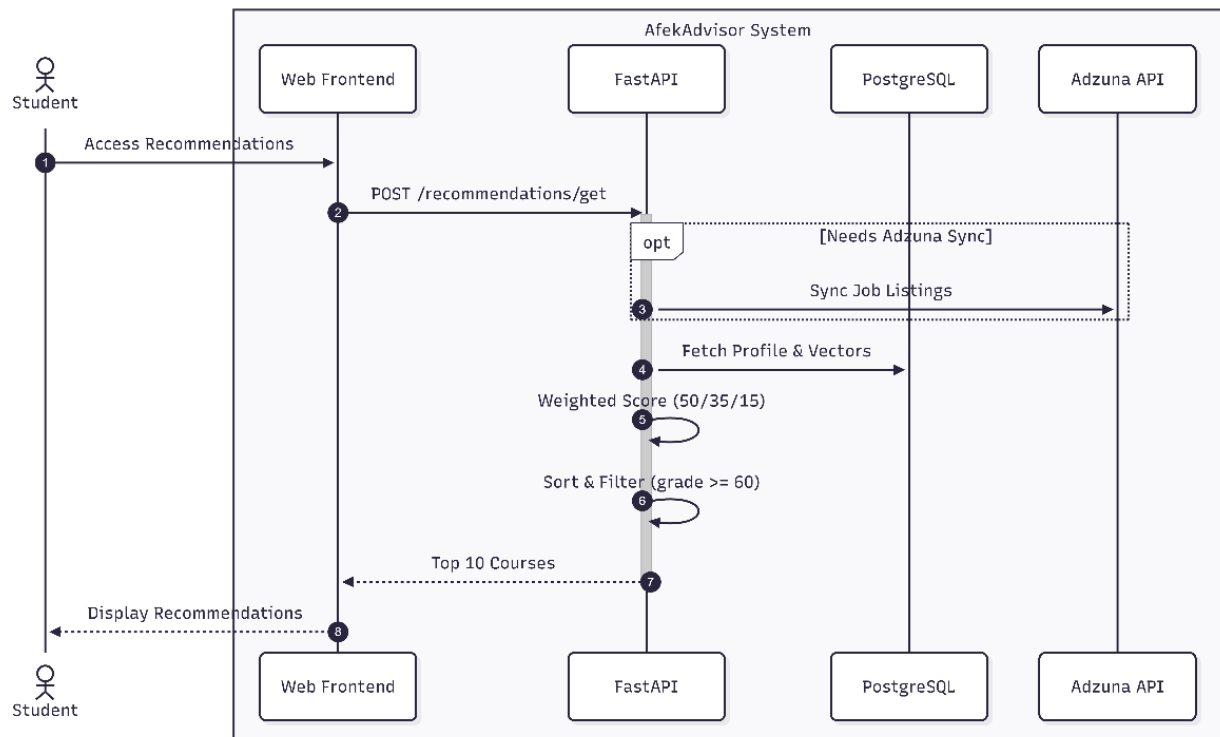
2) Academic Study Planning (Roadmap)



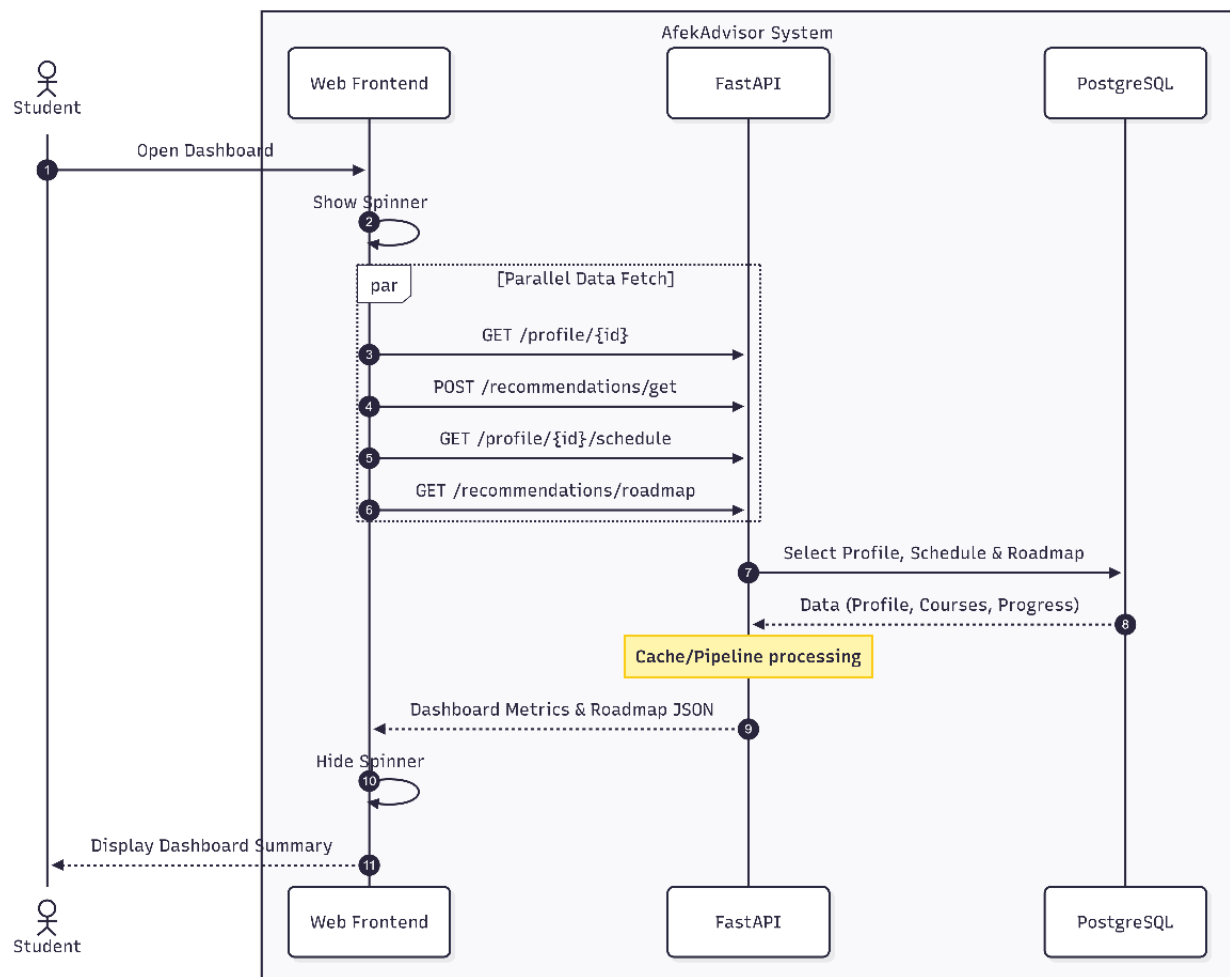
3) Course Filtering in the Course Explorer



4) Receiving Course Recommendations



5) Viewing Dashboard Data



4.6 Technology Stack and Tools

The AfekAdvisor system was developed as a Full-Stack application based on a Client-Server architecture, ensuring logical separation between the user interface (Frontend), the server (Backend), and the database.

The choice of technologies was aimed at ensuring optimal performance, high maintainability, and compatibility with computational tasks such as market data analysis and personalized recommendations.

The following table summarizes the tools and technologies chosen for each system component:

Layer	Technology	Rational and Technical Details
Frontend	React 19, TypeScript	Using React enables component-based development, facilitating interface maintenance. TypeScript adds a layer of Static Typing, preventing critical runtime errors when handling complex course data.
Backend	Python, FastAPI	FastAPI was chosen for its support of Asynchronous Programming (ASGI), allowing the server to handle hundreds of concurrent requests without blocking data processing. The system utilizes automatic API documentation (OpenAPI/Swagger), streamlining integration with the Frontend
Database	PostgreSQL, SQLAlchemy	A relational database ensuring data integrity (ACID compliance). Using an ORM (SQLAlchemy) allows writing clean Python code without the need for manual, complex SQL queries, and eases the maintenance of the course and student data schemas.
ML / IR	Scikit-learn, NumPy	Using TF-IDF enables converting free text (job postings) into numerical vectors. The calculation is performed using the NumPy library, which executes optimal matrix operations utilizing the CPU for rapid calculation of "Similarity" between a job and a course.
Infrastructure	Docker, APScheduler	Docker ensures the system runs in an identical environment on any server. APScheduler manages background tasks, such as weekly synchronization with the Adzuna API, updating the skills database without user intervention.

4.7 Architectural Patterns and Design Methods

The AfekAdvisor system consists of several subsystems that pass information between them, with data passing through multiple stations until a final response is generated. We placed high importance on choosing design patterns for the system. These architectural design patterns define the relevant parts in the subsystems, their connection types, and create stronger cohesion within a single subsystem.

MVC (Model-View-Controller)

In the MVC model, the system is divided into three main parts:

Model: Represents the business logic, data processing, and recommendation engine. It is responsible for TF-IDF calculations, managing the DAG, and saving data in PostgreSQL.

View: The Frontend (React) displaying information to the user, including the Dashboard, Course Explorer, and recommendation screens.

Controller: The entity managing the interaction. On the server, this is the API (FastAPI) receiving client requests, and on the client side, it is the HTTP Service executing server calls and managing the application state.

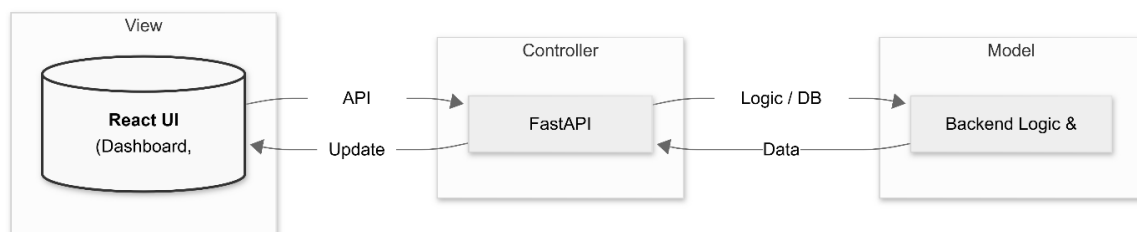


Figure 4.7.1 – MVC Architecture

Repository

The Repository pattern aims to centralize data storage in a single location away from the business logic. The subsystems (such as the recommendation engine or the roadmap service) access information centrally. In our system, users add data, rate courses, and view history, while the Repository provides a unified interface for all of these, ensuring data consistency.

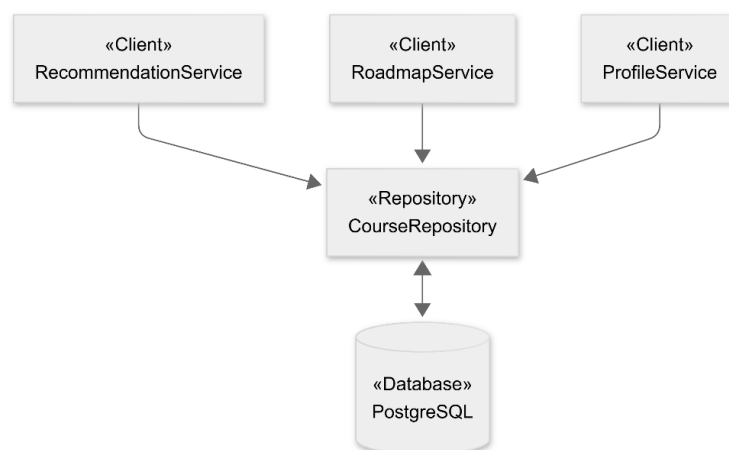


Figure 4.7.2 – Repository Pattern

Client/Server

The client/server pattern represents a central server operating autonomously and waiting for client requests. In our system, a remote server (Backend) communicates with user browsers. The server processes the student's personal profile and runs complex calculations, while the client is responsible for displaying data and surfacing personalized recommendations

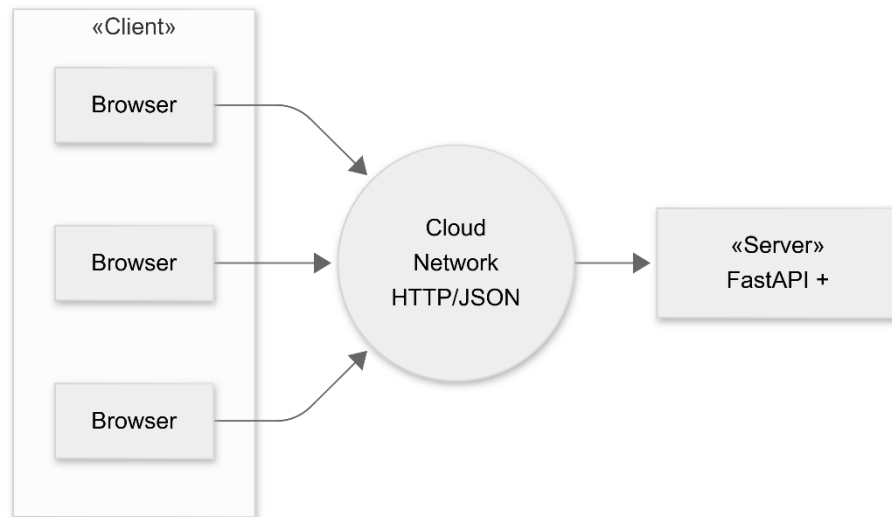


Figure 4.7.3 – Client/Server Architecture

Pipe & Filter

The employment market engine operates as a structured processing pipeline. Stage A fetches data, Stage B performs cleaning and removal of unnecessary characters, and Stage C performs vectorization. This structure enables robust logic - if one stage in the Pipeline fails, the exact location of the error can be identified without taking the entire system offline.

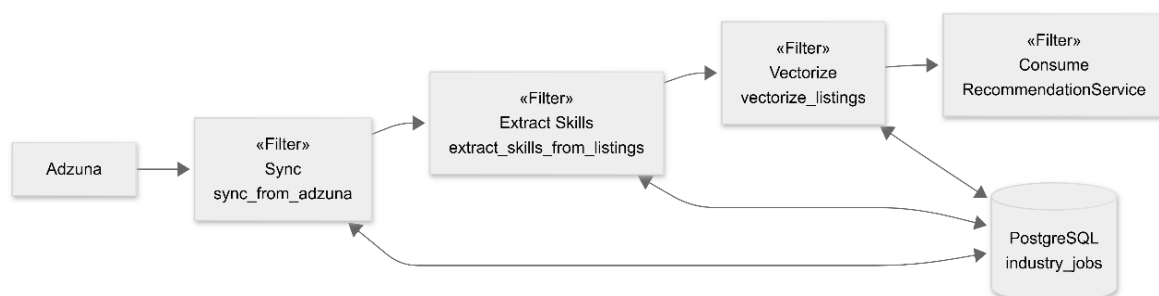


Figure 4.7.4 – Pipe & Filter Architecture

4.8 Development Methodology

System development was carried out using the Agile methodology, consisting of iterations (Sprints). In each sprint, we made progress in the user's ability to perform actions, starting from an MVP (Minimum Viable Product) to a full product including acceptance testing.

Sprint 1: Research and Infrastructure:

This sprint focused on setting up the environment and implementing core functionality. We set up the database, implemented the initial API for profile management, and conducted research on weighting grades and skills.

- Deliverables: Docker infrastructure, database models, and initial unit tests.

Task	Duration (Days)
Infrastructure and Docker setup	8
Building database schema (ORM)	2
MVP Development	11

Sprint 2: Interface and Integration:

In this sprint, the product became accessible to the user. We developed the Frontend in React, added authentication mechanisms (Auth), and began connecting the interface to the server. Comprehensive testing was conducted for edge cases and integration.

Task	Duration (Days)
System testing	3
Integration testing	4
User Interface (GUI) development	3

Sprint 3: Core Logic:

In this sprint, we completed the recommendation engine and complex algorithms. We focused on building the prerequisites DAG and semantic similarity calculations.

Task	Duration (Days)
Implementing recommendation logic	4.5
Building Roadmap engine (DAG)	4
Optimization	3

Sprint 4: Acceptance Testing, Security, and Optimization:

This sprint was dedicated to system Quality Assurance (QA), conducting repeated End-to-End (E2E) tests, and verifying recommendation quality. The system was optimized for response times, and final Acceptance Tests were performed.

Task	Duration (Days)
System and E2E testing	4
Acceptance and quality testing	2

5. System Structure & Architecture

5.1 High-Level System Overview

During the development and use of AfekAdvisor, the user interacts end-to-end with the system, which performs complex data processing in the background. The chosen architecture is Client/Server, maintaining a strict separation between the Presentation Layer (Frontend), the Business Logic, and the Persistence Layer (Backend).

1. **Endpoints (Boundary):** The student accesses the Web interface via a browser. The entry point is the Auth screen, followed by a personalized Onboarding process and the Dashboard. The system operates as a React-based SPA (Single Page Application), providing a seamless user experience.
2. **Data Transmission:** Communication is carried out via asynchronous HTTP/JSON requests. The Axios library manages data transfer, with each protected request authenticated using an Authorization: Bearer <JWT access_token> header. The token is issued at login and stored client-side.
3. **Data Processing:** The server (FastAPI) runs a multi-stage Pipeline: user authentication, academic profile retrieval, market data synchronization from external sources (Adzuna), calculating TF-IDF based recommendations, and building a Roadmap based on a dependency graph (DAG).
4. **Data Storage:** Information is stored in a PostgreSQL 15 database, utilizing aggressive Caching (both server-side and client-side) to reduce response times.
5. **External Component:** An automatic synchronization system (APScheduler) fetches employment market data weekly, ensuring real-time updated recommendations.

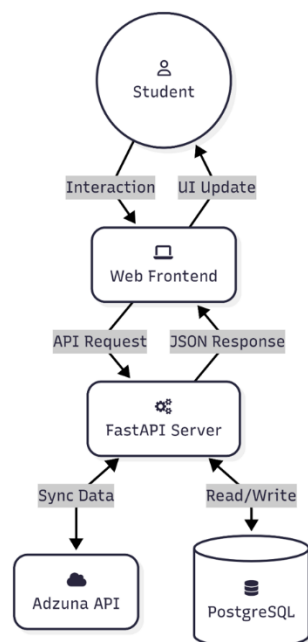


Figure 5.1.1 – High-Level Architecture

5.2 API Interface Specification

The API interface is the heart of communication between the system layers. It was designed with a RESTful approach to make the business and algorithmic logic accessible to the client interface.

API Functionality:

- **User and profile management:** Authentication, registration, and academic history management.
- **Course and track management:** Fetching course catalog, classification by year, and getting recommendations.
- **Planning management:** Fetching a prerequisite-based Roadmap.
- **Market synchronization:** Managing Pipeline processes against external sources.
- **Metadata:** Managing tracks, market roles, and technological dictionary synchronization.

The interface is based on the REST standard. Full documentation is available in Swagger. Swagger: <http://localhost:8000/docs>

Authentication (Auth)

Description	Method	Endpoint	Request	Response
Registration	POST	/register	{email, name, password}	{message}
Login	POST	/login	{email, password}	*(JWT + user profile)

Profile

Description	Method	Endpoint
Read Profile	GET	/profile/{student_id}
Update Profile	PUT	/profile/{student_id}
Single Course History	GET/POST/DELETE	/profile/{student_id}/history
Bulk History Update	POST	/profile/{student_id}/history/bulk

Courses and Recommendations

Description	Method	Endpoint
Course List	GET	/courses/
Yearly Mandatory	GET	/courses/yearly-mandatory
Reviews	GET/POST	/courses/{course_code}/reviews
Recommendations	POST	/recommendations/get
Roadmap	GET	/recommendations/roadmap

Market and Metadata

Description	Method	Endpoint
Adzuna Jobs Sync	POST	/jobs/sync
Extract Skills	POST	/jobs/extract-skills
Vectorize Listings	POST	/jobs/vectorize-listings
Average Target Vector	GET	/jobs/average-vector
Metadata	GET	/metadata/

5.3 Server-Side and Client-Side

Server-Side (Backend)

We chose the Python programming language, a popular software development language that adequately meets the project's requirements. Furthermore, Python has extensive support for numerous code libraries, especially those related to data processing and machine learning.

- **FastAPI:** The central server managing Routing and Authentication. It was chosen for being asynchronous and lightweight.
- **SQLAlchemy:** The ORM layer mediating between the logic and PostgreSQL, allowing the abstraction of complex queries.
- **NumPy & Scikit-Learn:** The libraries performing the "heavy lifting" converting text to vectors and calculating proximity between skills.
- **APScheduler:** A task scheduling engine critical for managing the market data Pipeline, running automatic weekly synchronization.
- **Infrastructure Libraries:** pydantic and typing were used for strong typing, and concurrency libraries for managing parallel processes under load.

Client-Side (Frontend)

Interface development was carried out using React 19 combined with TypeScript, providing a modular development environment that ensures strong typing and efficient component management. Using Vite as the primary Build tool ensured fast load times and agile development.

- **Communication Management (Axios):** For communication with the Backend, we chose the Axios library. We implemented a dedicated class (Service Layer) centralizing all server calls, ensuring consistency across the interface. This class uses Interceptors to automatically attach the JWT access token as Authorization: Bearer <token> on every outgoing request.
- **Interface with the Server:** The client operates according to the SPA (Single Page Application) architecture, allowing seamless transitions between system pages without page reloads. All requests to the FastAPI API are executed asynchronously, ensuring the interface remains responsive even during heavy server calculations.

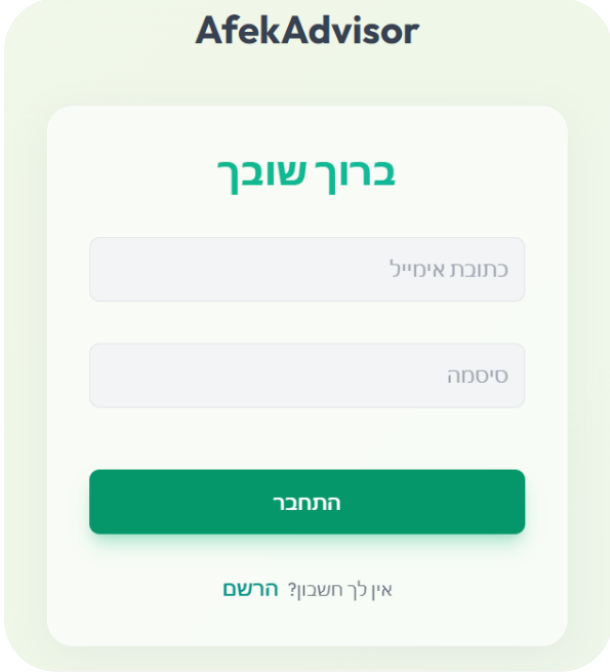
5.4 Graphical Interface

During the interface development process, emphasis was placed on improving the user experience through functionality, aesthetics, and accessibility. Functionality was achieved through prominent buttons for easy navigation. Aesthetics were addressed by incorporating attractive graphics, and accessibility was achieved through a simple structure that ensures quick and convenient access to the system.

Login Screen:

The user logs in with their credentials. If the email does not exist in the system or the details are incorrect, a red error message will be displayed. The registration screen can be accessed by clicking the "Register" button at the bottom.

Upon clicking "Login" - the user is redirected to Onboarding (first time) or the Dashboard

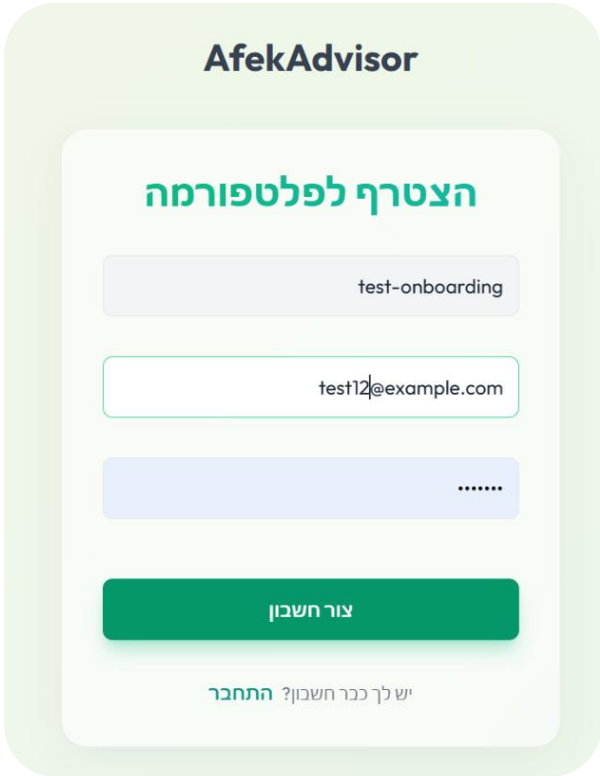


The login screen for AfekAdvisor features a light green background. At the top, the title "AfekAdvisor" is displayed in dark blue. Below it, the heading "ברוך שובר" (Baruch Shvur) is written in green. There are two input fields: the first is labeled "כתובת אימייל" (Email Address) and the second is labeled "סיסמה" (Password). A prominent green button labeled "התחבר" (Login) is positioned below the fields. At the bottom, a link "אין לך חשבון? הרשם" (Don't have an account? Register) is shown in green.

Registration Screen:

The user registers with their details. If the email already exists in the system, a red error message will be displayed. The login screen can be accessed by clicking the "Login" button at the bottom.

After clicking "Create Account" - the user is redirected to the onboarding process.



The registration screen for AfekAdvisor has a light green background. The title "AfekAdvisor" is at the top in dark blue. The heading "הצטרף לפלטפורמה" (Join the platform) is in green. It includes three input fields: a first field with the placeholder "test-onboarding", a second field with the placeholder "test12@example.com", and a third field for a password represented by dots. A green button labeled "צור חשבון" (Create Account) is located below the fields. At the bottom, a link "יש לך כבר חשבון? התחבר" (Do you already have an account? Login) is displayed in green.

onboarding:

ברוכים הבאים למערכת ההמלצות

בואו נגדיר את הפרופיל שלכם כדי שנוכל להתאים לכם את המסלול המושלם.

שלב 1: התואר שלך

בחר/י את מסלול הלימודים

מדעי המחשב

המשך לשלב הבא

ברוכים הבאים למערכת ההמלצות

בואו נגדיר את הפרופיל שלכם כדי שנוכל להתאים לכם את המסלול המושלם.

שלב 2: שנת לימודים

שנה א'

שנה ב'

שנה ג'

שנה ד'

המשך לשלב הבא

החזר

ברוכים הבאים למערכת ההמלצות

בואו נגדיר את הפרופיל שלכם כדי שנוכל להתאים לכם את המסלול המושלם.

שלב 4: היסטוריית קורסים

הזן/י את הקורסים שכבר עברת כדי שלא נמליץ עליהם שוב.

השלם קורסי חובה לשנה א'

חפש או בחר קורס

ציון

הוסף

0-100

הקלד שם או מספר קורס...

סיים ומעבר לזיכרון

החזר

ברוכים הבאים למערכת ההמלצות

בואו נגדיר את הפרופיל שלכם כדי שנוכל להתאים לכם את המסלול המושלם.

שלב 3: מסרות קריירה ומסלולים

ספרי לנו לאן את/ה מכוון/ת בתעשייה כדי שנוכל להמליץ על הקורסים הרלוונטיים ביותר.

תפקיד מבוקש בתעשייה

☐ Backend Engineer	☐ Full-Stack Developer
☐ Data Scientist	☐ Frontend Engineer
☐ Cyber Security Analyst	☐ DevOps Engineer
☐ Cloud Engineer	☐ Software Engineer
☐ Mobile Developer	☒ Machine Learning Engineer
☐ Systems Analyst	☐ QA Automation Engineer

מסלול התמחות

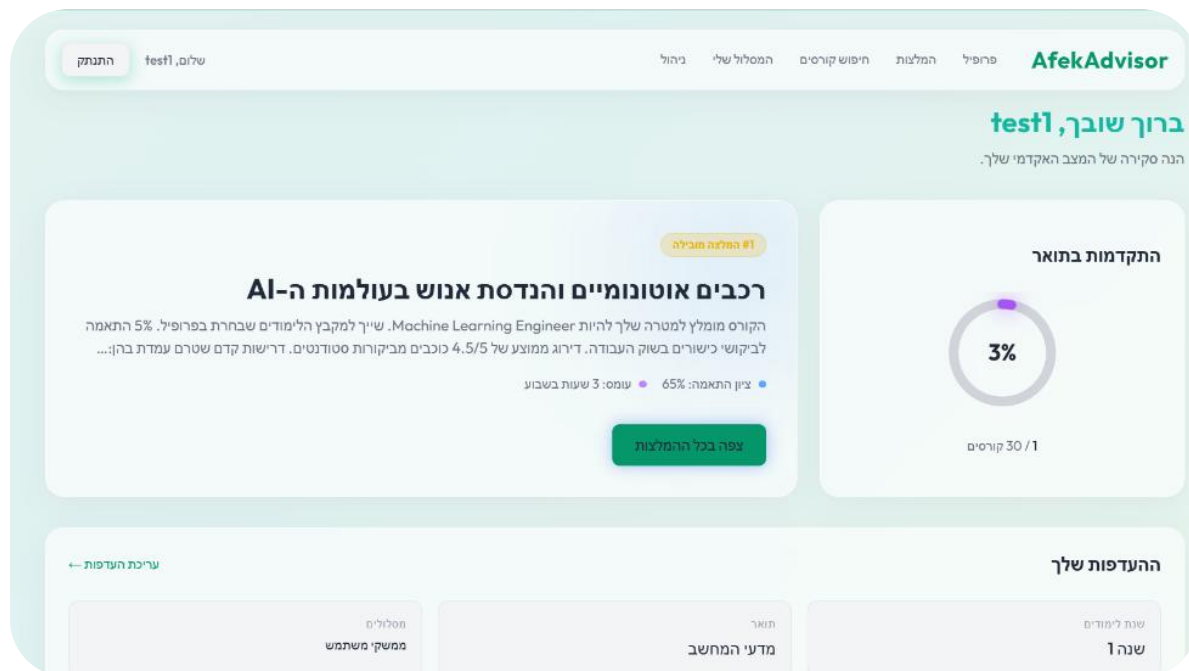
☒ למידת מכונה	☐ סייבר	☐ מחשקי משתמש
--	-----------------------------	-----------------------------------

המשך לשלב הבא

חזר

The system guides the new user through a 4-step onboarding process. At the top of the screen, a progress bar displays progress through the process. After selecting the study track, academic year, career goals and tracks, and course history, clicking the "Finish and proceed to dashboard" button saves the user's data in the database, and all recommendation calculations are performed on this data.

Home Screen (Dashboard):



The central screen provides a quick overview of academic status. By clicking "Edit Preferences," the user can navigate to the Profile screen, and by clicking "View all recommendations," they can access the Recommendations screen.

Profile:

The screenshot displays the 'Profile' page of the AfekAdvisor system. The page is divided into several sections:

- Header:** Includes the AfekAdvisor logo and navigation links: פרופיל (Profile), המלצות (Recommendations), חיפוש קורסים (Course Search), המסלול שלי (My Program), ניהול (Management), and התנתק (Logout). The user's name 'שלום, test1' is displayed.
- Personal Information:** A section titled 'פרופיל אישי' (Personal Profile) with the subtitle 'ניהול פרטים אקדמיים ומטרות קריירה' (Management of Academic Details and Career Goals). It contains three input fields: 'שנת לימודים' (Year of Study) with a dropdown menu, 'שנה א' (Year A)', and 'שנה ב' (Year B)'.
- Academic Details:** A section titled 'מסלול הלימודים' (Study Program) with the subtitle 'בחר/י את מסלול הלימודים' (Choose your study program). It features a dropdown menu for 'מדעי המחשב' (Computer Science) and four buttons for 'שנה ד' (Year D)', 'שנה ג' (Year G)', 'שנה ב' (Year B)', and 'שנה א' (Year A)'.
- Career Preferences:** A section titled 'מטרות קריירה ומסלולים' (Career Goals and Programs) with the subtitle 'נבחר/י מנוקש בגוגל' (Choose your preferred program). It contains a grid of 12 radio buttons for various roles: Backend Engineer, Full-Stack Developer (selected), Frontend Engineer, Data Scientist, Cyber Security Analyst, DevOps Engineer, Cloud Engineer, Software Engineer, Mobile Developer, Machine Learning Engineer, Systems Analyst, and QA Automation Engineer.
- Recommendations:** A section titled 'מסלול התמחות' (Specialization Program) with three radio buttons: 'למידת מכונה' (Machine Learning), 'סייבר' (Cyber), and 'ממשקי משתמש' (User Interfaces) (selected).
- Footer:** A blue button labeled 'עריכת ההעדפות' (Edit Preferences) and a note: 'שינוי ההעדפות ישפיע באופן מיידי על דירוג הקורסים המומלצים עבורך. לעמוד ההמלצות' (Changing preferences will immediately affect the ranking of recommended courses for you. Go to the recommendations page).

Displays all student's data. It can be edited by clicking the "Edit Preferences" button. Any change in preferences triggers an immediate update to the recommendations. The Recommendations screen can be accessed by clicking "Go to recommendations page".

Recommendations:

The screenshot displays the AfekAdvisor interface. At the top, there's a navigation bar with 'התנתק' and 'שלום, test1'. The main header 'המלצות חכמות' is followed by a sub-header 'בהתבסס על הקורסים שעברת ומטרות הקריירה, המערכת בנתה עבורך את המסלול המושלם.' Below this, two course recommendation cards are visible. The first card is for 'פיתוח בפלטפורמת WEB' (10266) with a 70% recommendation score. It lists skills like FastAPI, Node.js, React.js, TypeScript, JavaScript, CSS3, HTML5, Full-Stack Development, and REST APIs. The second card is for 'אפיון ממשקי משתמש' (10147) with a 69% recommendation score, listing skills like Requirements Gathering, Usability Testing, User Flows, Interactive Prototyping, and Figma. Each card includes a 'למה הקורס הזה?' section with a brief description and a 'דרג קורס' button.

The results of the recommendation algorithm calculation. The algorithm calculates a score for each course based on career goals, completed courses, specialization track, reviews, and required skills in the job market, displaying all this information on this screen. A course can be rated and reviewed by clicking the "Rate Course" button.

Course Rating:

The screenshot shows the 'פיתוח בפלטפורמת WEB' (10266) course rating page. It features a summary section with 'הנחות: גמישה', 'שעות: 3 שבועות', and 'ר"י: 3'. Below this, there are two main sections: 'דרישות קדם' (Prerequisites) and 'כישורים נרשמים' (Required Skills). The 'דרישות קדם' section includes 'פיתוח ממשקי משתמש' and 'תכנות מונחה עצמים'. The 'כישורים נרשמים' section lists various technologies like CSS3, HTML5, Full-Stack Development, Node.js, React.js, TypeScript, JavaScript, REST APIs, and FastAPI. A 'ביקורות סטודנטים' (Student Reviews) section is at the bottom, showing a 5-star rating and a text input field for reviews. A 'פרסם ביקורת' button is present, along with a checkbox for 'פרסם באנונימיות'.

On this screen, reviews can be added with an option for anonymous writing, courses can be rated, and previous reviews for the course can be read.

Course Search (Explorer):

The screenshot shows the 'AfekAdvisor' web application. At the top, there's a navigation bar with links: 'התנתק' (Logout), 'שלום, festl', 'ניהול', 'המסלול שלי', 'חיפוש קורסים' (Course Search), 'המלצות' (Recommendations), 'פרופיל' (Profile), and the 'AfekAdvisor' logo. The main heading is 'חיפוש קורסים' (Course Search) with a subtitle 'סנן לפי מקבץ לימודים' (Filter by course cluster). Below this are four filter buttons: 'מקבץ למידת מכונה' (Machine Learning cluster), 'מקבץ סיבר' (Cyber cluster), 'מקבץ ממשקי משתמש' (User Interface cluster), and 'כל המקבצים' (All clusters). A search bar contains the text 'חפש לפי שם או קוד...'. Below the search bar are three checkboxes: 'ללא נוכחות חובה' (No mandatory attendance), 'עומס נמוך' (Low workload), and 'דרישות קדם מולאו' (Prerequisites met). The main content area displays three course cards. Each card has a title, a table with 'מכחות' (Credits), 'שעות בשבוע' (Hours per week), and 'ג'י' (Grade), and a 'דרישות קדם' (Prerequisites) section. The first card is for course 10243, 'רשתות ניורונים לראייה ממוחשבת' (Neural Networks for Computer Vision), with 3 credits, 3 hours per week, and a grade of 3. The second card is for course 10240, 'רשתות ניורונים ולמידה עמוקה' (Neural Networks and Deep Learning), with 3 credits, 3 hours per week, and a grade of 3. The third card is for course 10233, 'פיתוח מאובטח' (Secure Development), with 3 credits, 3 hours per week, and a grade of 2.5. Below these cards are three more course IDs: 90102, 10403, and 10401.

On this screen, the user can view all the courses in their degree and filter them by text, clusters, attendance type, workload, and all courses for which the user has met the prerequisites and can register for.

Clicking on a Course:

The screenshot shows the details page for course 10233, 'פיתוח מאובטח' (Secure Development). The page has a title bar with the course ID '10233' and a close button. Below the title, there's a table with 'מכחות' (Credits), 'שעות בשבוע' (Hours per week), and 'ג'י' (Grade). The 'דרישות קדם' (Prerequisites) section lists 'תכנות מונחה עצמים' (Object-oriented programming). The 'כישורים נרכשים' (Skills acquired) section lists 'Vulnerability Prevention', 'Secure Coding', 'API Security', and 'Buffer Overflow Prevention'. Below this are two buttons: 'קרא / כתוב ביקורת' (Read / Write review) and 'הוסף להרשימה שלי' (Add to my list). The 'ביקורות סטודנטים' (Student reviews) section shows two reviews: one from 'user1' with a 5-star rating and one from 'Anonymous' with a 2-star rating. The page also includes a footer with the text 'הרבה מסלולים. קשה לעמוד בלוח הזמנים.'

Clicking "Add to my list" will add the course to the user's course history, and clicking "Read/Write review" gives the user the option to rate the course and write a review, including an option for anonymity.

My Track (Roadmap):

התנתק

שלום, test1

ניהול

המסלול שלי

חיפוש קורסים

המלצות

פרופיל

AfekAdvisor

המסע האקדמי שלי

מפת דרכים לסיום התואר — חובות, בחירה חופשית וקורסים מומלצים

3% הושלם

0/23 חובה

0/1 סמינר

1/6 בחירה חופשית

0/23 קורסי חובה

קורסים שעברתי

1 קורסים רשומים.

1 שנה א' - סמסטר א'

עתידי

10016

מבוא למדעי המחשב

חובה: 4.5 נ"ז

עתידי

90905

אלגברה ליניארית 1

חובה: 5 נ"ז

עתידי

90901

חשבון דיפרנציאלי ואינטגרלי 1

חובה: 5 נ"ז

עתידי

90926

התנתק

שלום, test1

ניהול

המסלול שלי

חיפוש קורסים

המלצות

פרופיל

AfekAdvisor

0/1 סמינר

1/6 בחירה חופשית

0/23 קורסי חובה

קורסים שעברתי

הוסף קורסי חובה לפי שנה

קורסי חובה שנה א'

קורסי חובה שנה ב'

קורסי חובה שנה ג'

חפש או בחר קורס

הקלד שם או מספר קורס...

ציין

הוסף

0-100

▼

קורס

ציין

100

✖

10233 פיתוח מאובטח

שמור

33

התנתק

שלום, fest1

ניהול

המסלול שלי

חיפוש קורסים

המלצות

פרופיל

AfekAdvisor

חובה 3 כ"ז

הושלם

10233

פיתוח מאובטח

בחירה 2.5 כ"ז

עתידי

19101

מבוא לבינה מלאכותית

חובה 2.5 כ"ז

עתידי

90954

אלגברה ליניארית 2

חובה 5 כ"ז

מומלץ

10147

אפיון ממשקי משתמש

בחירה 4 כ"ז

מומלץ

90105

תכנות מונחה עצמים

בחירה 4.5 כ"ז

שנה ב' - סמסטר ב' 4

עתידי

10120

תכנון וניתוח אלגוריתמים

חובה 5 כ"ז

עתידי

10139

מודלים חישוביים

חובה 5 כ"ז

עתידי

10013

תקשורת מחשבים

חובה 3.5 כ"ז

מומלץ

90101

מבוא למדעי המחשב

בחירה 4.5 כ"ז

מומלץ

90116

תקשורת מחשבים

בחירה 3.5 כ"ז

עתידי

10303

מערכות הפעלה

חובה 3.5 כ"ז

מומלץ

10220

פיתוח משחקים

בחירה 2.5 כ"ז

מומלץ

90111

מבוא להסתברות

בחירה 3.5 כ"ז

This screen displays the student's complete study map - mandatory courses, free electives, and seminars. It integrates the recommendations based on the recommendation algorithm into the student's study map and builds a comprehensive study plan from the beginning of the degree to its end.

Management:

התנתק

שלום, test1

ניהול

המסלול שלי

חיפוש קורסים

המלצות

פרופיל

AfekAdvisor

ניהול מערכת

הוספת קורס

קוד קורס (7 ספרות)
לדוגמה: 1234567

שם הקורס

שם הקורס

קטגוריה
בחירה

עומס שבועי (שעות)
3

נקודות זכות (נ"ז)
3

כישורים (מפורדים בפסיק)
לדוגמה: Python, SQL, אלגוריתמים

דרישות קדם (סקסט חופשי)
לדוגמה: מבוא למדעי המחשב, מבני נתונים

צור קורס

התנתק

שלום, test1

ניהול

המסלול שלי

חיפוש קורסים

המלצות

פרופיל

AfekAdvisor

צור קורס

קורסים (106)

חיפוש לפי שם/קוד

10401 Data Structures and Algorithms נכחות שעות נ"ז גמישה 8 3 מחיקה עריכה	10243 רשתות ניוונים לראייה ממוחשבת נכחות שעות נ"ז גמישה 3 3 מחיקה עריכה	10240 רשתות ניוונים ולמידה עמוקה נכחות שעות נ"ז גמישה 3 3 מחיקה עריכה	10233 פיתוח מאובטח נכחות שעות נ"ז גמישה 3 2.5 מחיקה עריכה
10208 פיתוח ממשקי משתמש נכחות שעות נ"ז גמישה 4 4 מחיקה עריכה	10127 בסיסי נתונים נכחות שעות נ"ז גמישה 3 3 מחיקה עריכה	90102 מתמטיקה בדידה נכחות שעות נ"ז גמישה 6 5 מחיקה עריכה	10403 Database Systems נכחות שעות נ"ז גמישה 5 3 מחיקה עריכה
10234 אבטחת מובייל נכחות שעות נ"ז גמישה מחיקה עריכה	10405 Cloud Architecture נכחות שעות נ"ז גמישה מחיקה עריכה	10224 מבוא לראייה ממוחשבת נכחות שעות נ"ז גמישה מחיקה עריכה	10220 פיתוח משחקים נכחות שעות נ"ז גמישה מחיקה עריכה

10240
✕

עריכת קורס

שם הקורס

רשתות נוירונים ולמידה עמוקה

קטגוריה

3

▼

נקודות זכות (נ"ז)

3

דרישות קדם (טקסט חופשי)

מבוא להסתברות, למידת מכונה (במקביל)

רשתות נוירונים ולמידה עמוקה

עומס שבועי (שעות)

3

כישורים (מופרדים בפסיק)

ks, CNN, Transformers, Overfitting Prevention

ביטול
שמור שינויים

התנתק test1, שלום,

ניהול
חיפוש קורסים
המסלול שלי
המלצות
פרופיל

AfekAdvisor

נכחות

גמישה

3

שעות בשבוע

ניחול

מחיקה

נכחות

גמישה

3

שעות בשבוע

עריכה

נכחות

גמישה

3

שעות בשבוע

מחיקה

נכחות

גמישה

3

שעות בשבוע

עריכה

#	שם	אימייל	תפקיד
1	test	test@example.com	סטודנט
2	Hila	hindihila@gmail.com	סטודנט
3	user1	pipeline.reviewer1@seed.local	סטודנט
4	user2	pipeline.reviewer2@seed.local	סטודנט
5	user3	pipeline.reviewer3@seed.local	סטודנט
6	test1	test1@example.com	מנהל
10	דני לדוגמא	dani.demo2025@test.com	סטודנט

The management screen is available only to administrators and allows for the management of courses in the database. Courses can be added, existing courses can be edited, and user permissions can be managed.

5.5 Algorithmics and System Logic

The core engines of AfekAdvisor are not based on guesswork, but on deterministic, multi-stage processes. The system is designed to solve two complementary problems: the first is long-term planning, constructing a study plan for three years (six semesters) while ensuring compliance with prerequisites and a balanced academic workload; the second is short-term recommendation, ranking relevant courses for the student based on their study track, job market demands, and quality ratings.

The system operates through two main services:

1. **RoadmapServiceImpl**, responsible for planning the entire degree.
2. **RecommendationServiceImpl**, responsible for optimizing courses for the upcoming semester.

The Prerequisite Problem and the DAG Model

Planning an academic degree is a classic constraint problem, as an advanced course cannot be taken before completing its foundational courses. To represent this system, AfekAdvisor builds a directed graph where the nodes are courses, and the edges are the "prerequisite - dependent course" relationship. When there are no cycles in the graph, it is called a DAG (Directed Acyclic Graph). This structure ensures there is a valid study order that can be calculated using topological sorting.

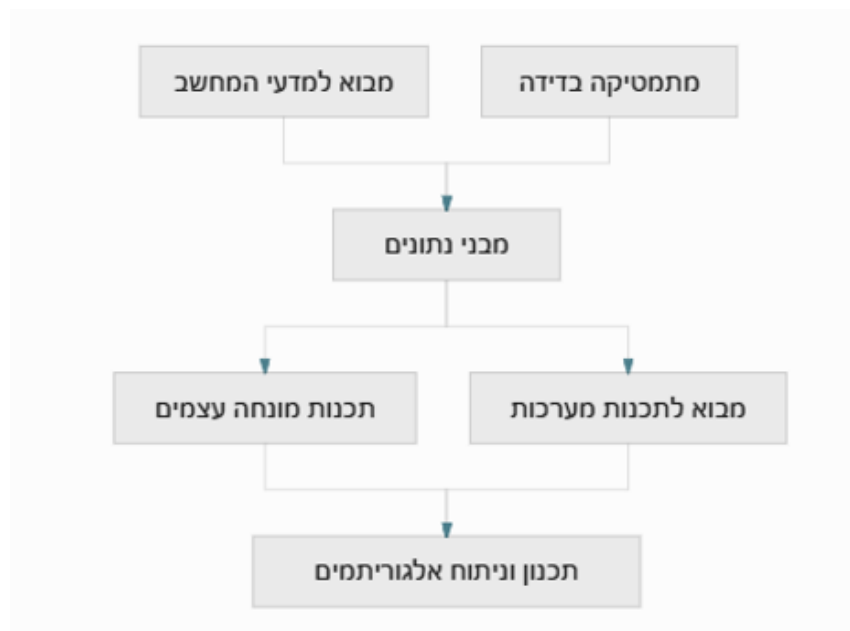


Figure 5.5.1 – Example: A chain of prerequisite requirements among core Computer Science courses.

Topological Sorting: Kahn's Algorithm

After building the DAG, the system uses Kahn's algorithm to determine the study order. The algorithm operates with an efficiency of $O(V + E)$ and includes five steps:

First, loading the courses from the catalog.

Second, calculating the In-Degree for each course.

Third, initializing a queue with all courses having an In-degree of zero.

Fourth, popping courses from the queue and updating their dependents.

Finally, mapping the courses into semesters while enforcing Year-floor rules (e.g., Year A to semesters 1-2, and so on) and balancing the semester workload.

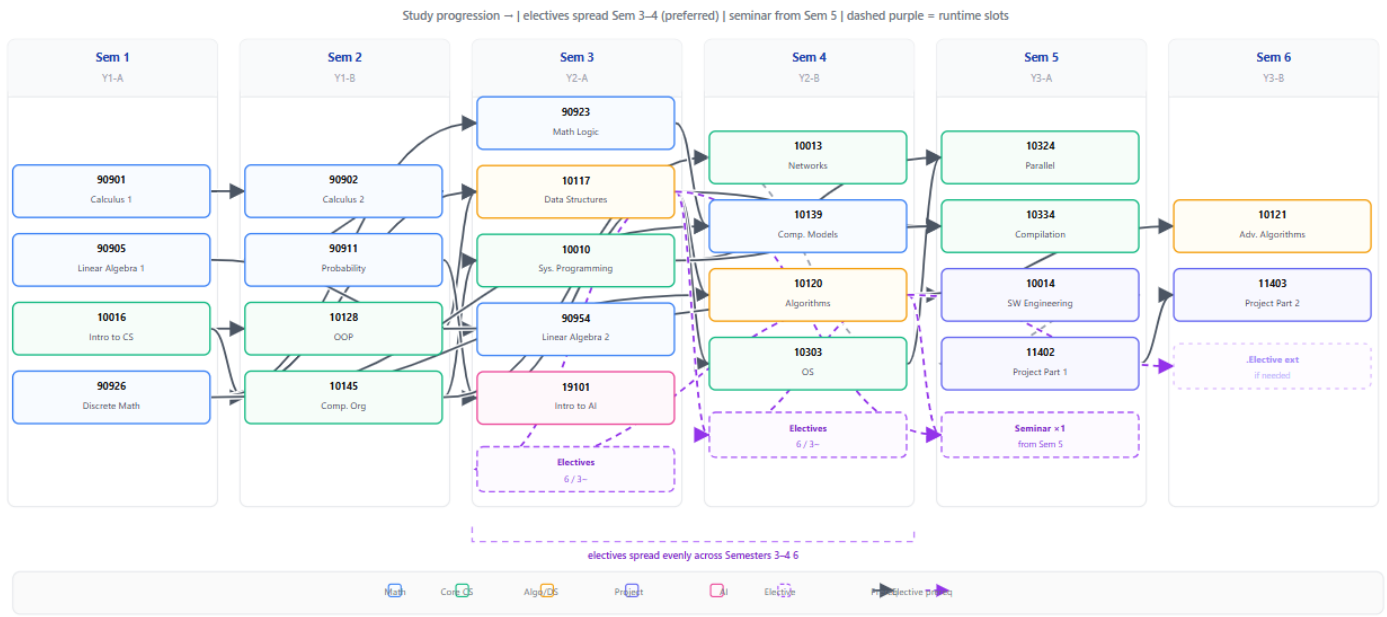
Injecting Electives and Seminars

After placing the mandatory courses, the system activates a mechanism to inject elective courses. In this stage, the system calls the recommendation engine to fetch the most suitable courses for the student, filters out already completed courses, and expands the DAG according to the prerequisites of the newly added electives. If missing courses are found, the system reruns the mapping steps to ensure a complete, consistent, and valid study plan.

Semester Constraints and Final Output

To ensure a reasonable workload, the system enforces a cap of up to 8 courses per semester, aiming for a balance of 7 courses. The final output of the process is a structured JSON object containing the study plan for all six semesters, including course details and their progress status. This data is cached for 300 seconds to optimize system performance when navigating between the Dashboard and the academic history screens.

A — Prerequisite DAG



B — Kahn topological sort pipeline



C — Elective & seminar injection



Figure 5.5.2 – Academic Planning Process:
From the DAG structure and constraints (A), through the topological sorting steps of Kahn's algorithm (B), to the dynamic injection of electives and seminars (C).

Recommendation Engine: Weighted Score and TF-IDF

The recommendation engine operates by calculating a weighted score composed of three metrics: study track suitability (50%), job market suitability (35%), and student quality rating for the course (15%). To compare course content with job requirements, the system uses a TF-IDF model that converts text into numerical vectors. The model assigns a high weight to unique technological terms and less to general words. By calculating Cosine Similarity, the system measures the proximity between the student's skills and market demands.

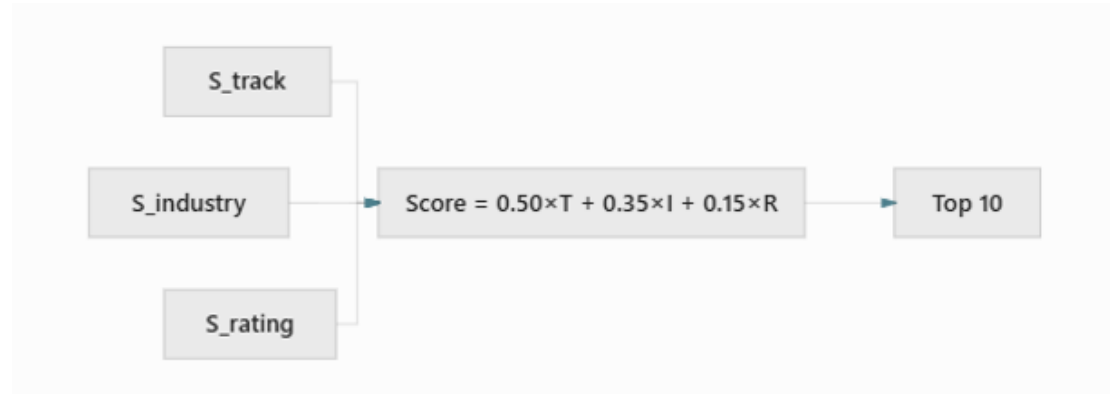


Figure 5.5.3 - Combined Weighting Formula: Balancing the study track, job market demands, and student ratings

Market Data Pipeline (Pipe & Filter)

To bridge the gap between academia and changing industry demands, the system runs an automated Pipeline that processes real-time job data without manual intervention. The process consists of four sequential stages (Pipe & Filter):

1. **Sync:** In the first stage, the system performs a Scheduled Task to pull job postings via the Adzuna API. The data is saved in the industry_jobs table, which includes the full job description, technological requirements, and geographic location.
2. **Extract:** Job descriptions undergo a cleaning process and are scanned against a technological Taxonomy. In this stage, the system filters out linguistic 'noise' and extracts keywords such as "React", "Python", or "System Architecture", which constitute the technological DNA of the job.
3. **Vectorize:** Using our defined TF-IDF model, each job is converted into a numerical Feature Vector.
4. **Consume:** In the final stage, the vectors of all relevant jobs are calculated into a single industry_target_vector – this is the average vector representing the "ideal job profile" in the industry for a given role. This vector is saved in the database.

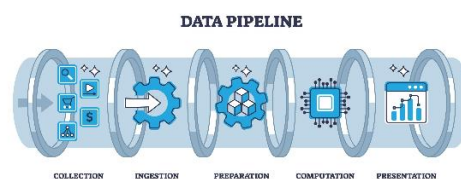


Figure 5.5.4 – Market Data Processing: Fetching data from an external API, extracting skills, converting to numerical vectors, and creating an ideal market profile.

Recommendation Workflow Summary

When a student requests a recommendation, the process occurs as follows:

1. **Identification and Return:** The system checks the Cache for previous results to provide an immediate response.
2. **Data Processing:** If a new calculation is required, the student's data is loaded, and synchronization with the job market (Adzuna) is performed.
3. **Calculation:** Running the weighted model for each potential course: Track Score (50%) + Job Match Score (35%) + Student Reviews Score (15%).
4. **Validation (Constraints):** The system checks prerequisites. If a recommended course requires another untaken course, the algorithm automatically "injects" the prerequisite course into the student's study schedule.
5. **Ranking and Display:** The system sorts all courses from highest to lowest score and presents the top 10 best courses for the students, along with an explanation of why they were chosen.

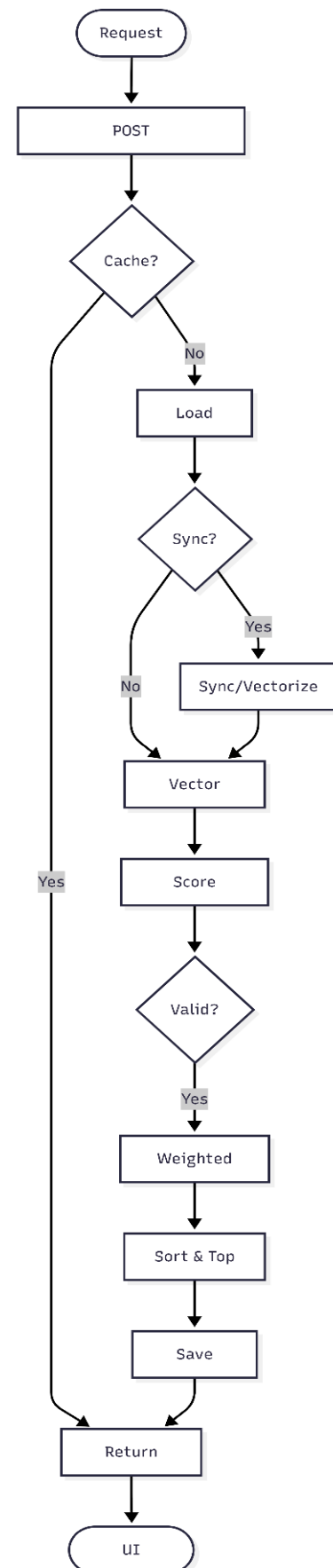


Figure 5.5.5 – Recommendation Flowchart: From Cache filtering to the final ranking of recommended courses.

Data Infrastructure and Algorithm Implementation

The business logic is separated into defined components:

RecommendationServiceImpl for calculating recommendations, RoadmapServiceImpl for graph management, and JobPipelineServiceImpl for maintaining market data.

The TF-IDF algorithm is implemented within LocalEmbeddingServiceImpl, while the storage and retrieval of data are executed through the Repository layer against the PostgreSQL database.

This separation ensures that the system remains scalable and maintainable over time, providing a rapid and accurate response to all the students' academic and career planning needs.

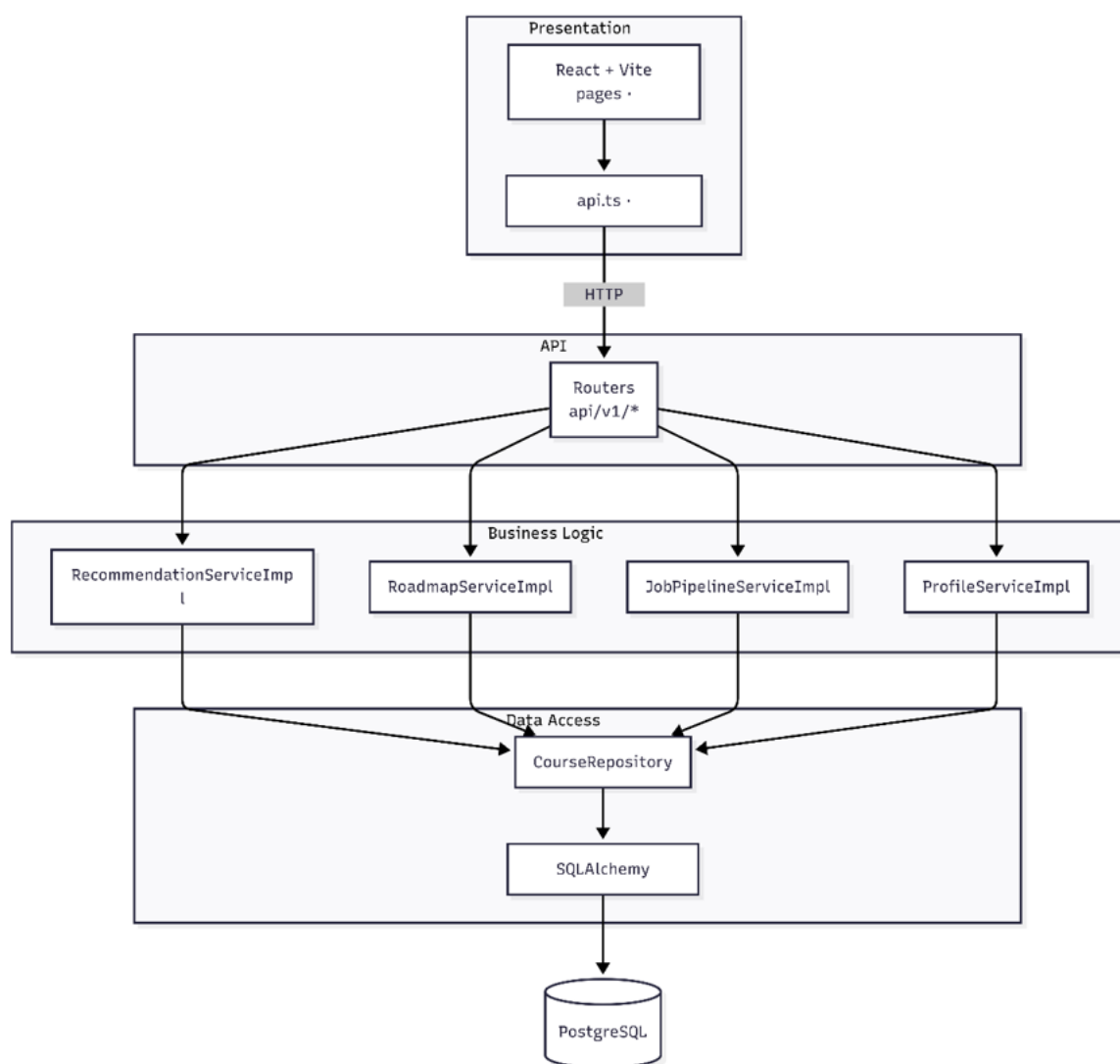


Figure 5.5.6 – System Layer Structure: Separating business logic, data access layer, and the database.

6. Execution and Implementation

6.1 Development and Runtime Environment

The work environment was designed to ensure stable operation from the development phase to cloud deployment, utilizing Docker technology.

Local Development Environment

Development was carried out on a local machine, separating the database from the application components. For convenience, the database (PostgreSQL) was run using Docker Compose, while the Backend and Frontend ran natively on the machine. This approach allowed for rapid and convenient development without reliance on complex configurations.

Production Environment (AWS EC2):

The system was deployed on a virtual cloud server (AWS EC2). Unlike the development environment, all system components—the server, database, and interface- are executed within containers managed by Docker Compose.

- **Management and Maintenance:** The system was configured with an automatic restart policy (restart: always), ensuring that services autonomously recover in the event of failure.
- **Access:** The system is publicly accessible via the server's public IP.

Data Synchronization and Automation:

The system manages the updating of market data through two channels:

- **Background Task (Cron):** Utilizing APScheduler to run a weekly task for updating Job Roles.
- **Dynamic Update:** When requesting recommendations or updating a profile, the Backend triggers a Pipeline that performs job fetching (Adzuna), skill extraction, and vectorization. This process ensures that the system's recommendations constantly reflect current market demands without requiring manual user intervention.

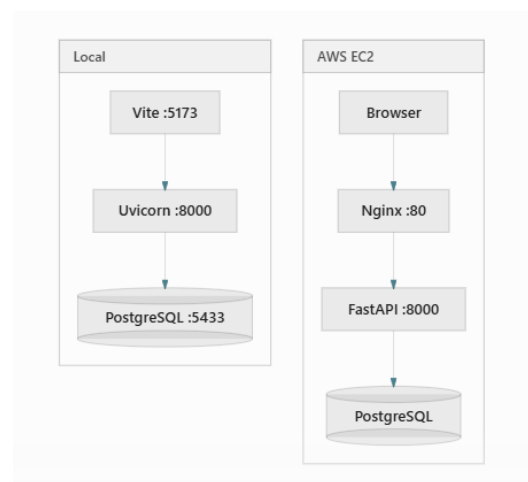


Figure 6.1.1 – Deployment Scheme: A comparison between the local development environment and the production environment on an AWS EC2 server.

6.2 Software Testing

This section describes the functional tests conducted on the system to ensure its proper operation.

Authentication and Onboarding Tests (Auth & Onboarding)

Description	Test	Result
Creating an account with a valid email	New registration	Pass
Duplicate prevention	Registration attempt with an existing email	"Email already registered"
Incorrect login	Entering an incorrect password	"Incorrect email or password"
Onboarding process	Completing 4 steps (degree, year, track, history)	onboarding_completed: true

Recommendation Logic and Course Explorer

Description	Test	Result
Top 10 limit	Requesting course recommendations	Up to 10 courses are displayed
Course filtering	A course already passed with a grade ≥ 60	Does not appear in recommendations
Score details display: track, industry, rating	Score breakdown	Displayed on the course card
Course Explorer	Filtering courses by category	Displaying only relevant courses
Sending a review	Rating a course and writing free text	Review is saved in the DB

End-to-End User Scenarios

Description	Scenario	Result
New student journey	Registration -> Full Onboarding -> Receiving initial recommendations	Pass
Study track planning	Selecting specialization -> Reviewing prerequisites -> Adding courses to the plan	Pass
Market data update	Running the Pipeline process -> Verifying skills update	Pass
Study and work integration	Filtering courses by time constraints (hours/attendance) -> Receiving an adjusted schedule	Pass

Error and Infrastructure Scenarios

Description	Test	Result
Access without Auth	Accessing a protected route without a Token	Redirect to Auth
API call without Authorization header	Request without Authorization header on protected endpoint	401 Unauthorized
Market Sync (Cron)	Checking the weekly APScheduler task execution	Sync successful at 01:00
Filter error	Searching for a non-existent course	"No results found" message

7. Comparing Planning vs. Execution

This chapter presents a comparative overview between the objectives defined during the initial planning phase and the final implemented system - AfekAdvisor. The chapter analyzes the gaps, details the changes made during the development process, and discusses the engineering lessons learned.

7.1 Responsiveness, Adherence to Planning, and Changes

In the early stage of planning, detailed functional and technical goals were defined, aimed at addressing the difficulties Computer Science students face when choosing study tracks. The established system successfully meets the vast majority of these goals: secure user management, a multi-step Onboarding process, a smart recommendation engine, the generation of DAG-based study plans (Roadmap), and a Course Explorer for searching and filtering courses.

During development, we made several conscious adjustments to the original engineering document:

- **Recommendation Algorithm:** We shifted to using a local TF-IDF model via scikit-learn instead of a cloud model. This change reduced reliance on external networks and enabled offline calculation.
- **Market Data Management:** The external Adzuna API updates market roles on a scheduled basis, but the skill recommendation extraction logic runs selectively only when a student updates their profile, to prevent system overload.
- **Runtime Architecture:** For rapid development, the Frontend and Backend were run natively in the workspace, while in the production environment (AWS EC2), the entire system runs securely and systematically within containers.

Components like a dedicated Mobile app or direct integration with internal college systems were pre-defined as out of scope to allow us to focus on the independent recommendation engine.

7.2 Difficulties Encountered During Development

During development, we encountered several major challenges, the most prominent being performance:

The Performance Challenge and Cache :

One significant problem we faced was an overly long response time (over 30 seconds) from the recommendation engine for each request, due to heavy vector calculations performed in real-time. The primary solution was implementing a sophisticated Caching layer that saves the calculation results for each student.

Thanks to the Cache, we significantly shortened the response time for repeated requests, making the system much more user-friendly.

Integration with an External API :

Working with employment market data (Adzuna) presented rate limit constraints and network dependencies. We overcame this by implementing a Fallback mechanism that displays pre-saved data in case the external service is unavailable.

7.3 Discussion and Lessons Learned

Developing the AfekAdvisor system demonstrated that the success of an engineering project depends first and foremost on preliminary architectural planning. The main lesson we learned is that properly planning the data model (ERD) saves weeks of rework.

Beyond that, our experience led to several additional engineering insights:

Code Structure Planning:

We adhered to a clean code structure separating the Client (React) and the Server (FastAPI).

The server was divided into dedicated layers: Routers for request management, Services for business logic, and Repositories for data access.

Using the Repository Pattern allowed us to easily replace and improve the recommendation algorithm without breaking the user interface (UI), making the system much more stable and maintainable.

Working with Swagger:

Using Swagger for API documentation was a central work tool. It not only provided us with a convenient "map" of all Endpoints during development but also served as a quick manual testing tool, allowing us to verify that the Backend returns data in the expected format even before connecting it to the Frontend.

Development Methodology:

Utilizing the Agile method helped us focus on the core, receive rapid feedback from the field, and make real-time corrections.

8. Future Development

8.1 Project Summary and Conclusions

The AfekAdvisor project constituted a significant engineering journey for us, translating a genuine need of Computer Science students into a working technological system. The system we developed bridges the gap between academic knowledge acquired during the degree and the dynamic demands of the job market, providing personalized tools for the student in their career planning process.

The software was developed using advanced technologies (such as React and FastAPI), providing a precise and high-quality technological solution tailored to Afeka CS students.

Conclusions from the Process:

- Meticulous architectural planning, including layer separation, is the key to building a stable system.
- Working with complex data and external APIs requires building Fallback solutions and flexible data structures, ensuring functional continuity even under uncertainty.
- Any final product loses its value if it is not made accessible through a simple and intuitive interface for the student.

8.2 Recommended Future Development

Currently, the system provides a complete and cohesive solution. However, we have identified several technological directions that could elevate it to the next level:

- **Dynamic Schedule:** Developing a module for managing a personal schedule. The system will allow students to import their chosen mandatory and elective courses and display them in a focused weekly view, alerting them of course conflicts and recommending an optimal semester for each course.
- **Degree Calculator Integration:** The system will automatically calculate the accumulation of credits required to complete the degree and the weighted Grade Point Average (GPA).
- **Smart Support AI Chatbot:** Integrating a language-model-based Chatbot to serve as an academic advisor available 24/7.
- **Improving the Recommendation Algorithm:** Implementing advanced Natural Language Processing models (such as sentence-transformers) instead of TF-IDF, to achieve a deeper understanding of the connection between syllabi and actual job requirements.
- **Integration with College Systems:** Automatic connection to the college's information systems to import the student's grade and course history without requiring manual data entry.
- **Response Time Improvements:** The current system relies on basic Caching. Future development will focus on transitioning to Redis as a central, distributed Cache server, allowing for more efficient management of recommendations for many concurrent users and preventing costly repeated calculations.

9. Bibliography

Academic Research and Literature Review:

1. Ajanovski, V. V. (2019). A visual domain exploration tool for personalized student advice and guidance. Proceedings of the 2019 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE).
2. Atalla, S., Daradkeh, M., Gawanmeh, A., et al. (2023). An Intelligent Recommendation System for Automating Academic Advising Based on Curriculum Analysis and Performance Modeling. *Mathematics*, 11(5), 1098.
3. Ovtšarenko, O. (2023). Generation of a learning path in e-learning environments: literature review. *New Trends in Computer Sciences*, 1(1), 32–50.

Technologies and Interfaces (Official Documentation):

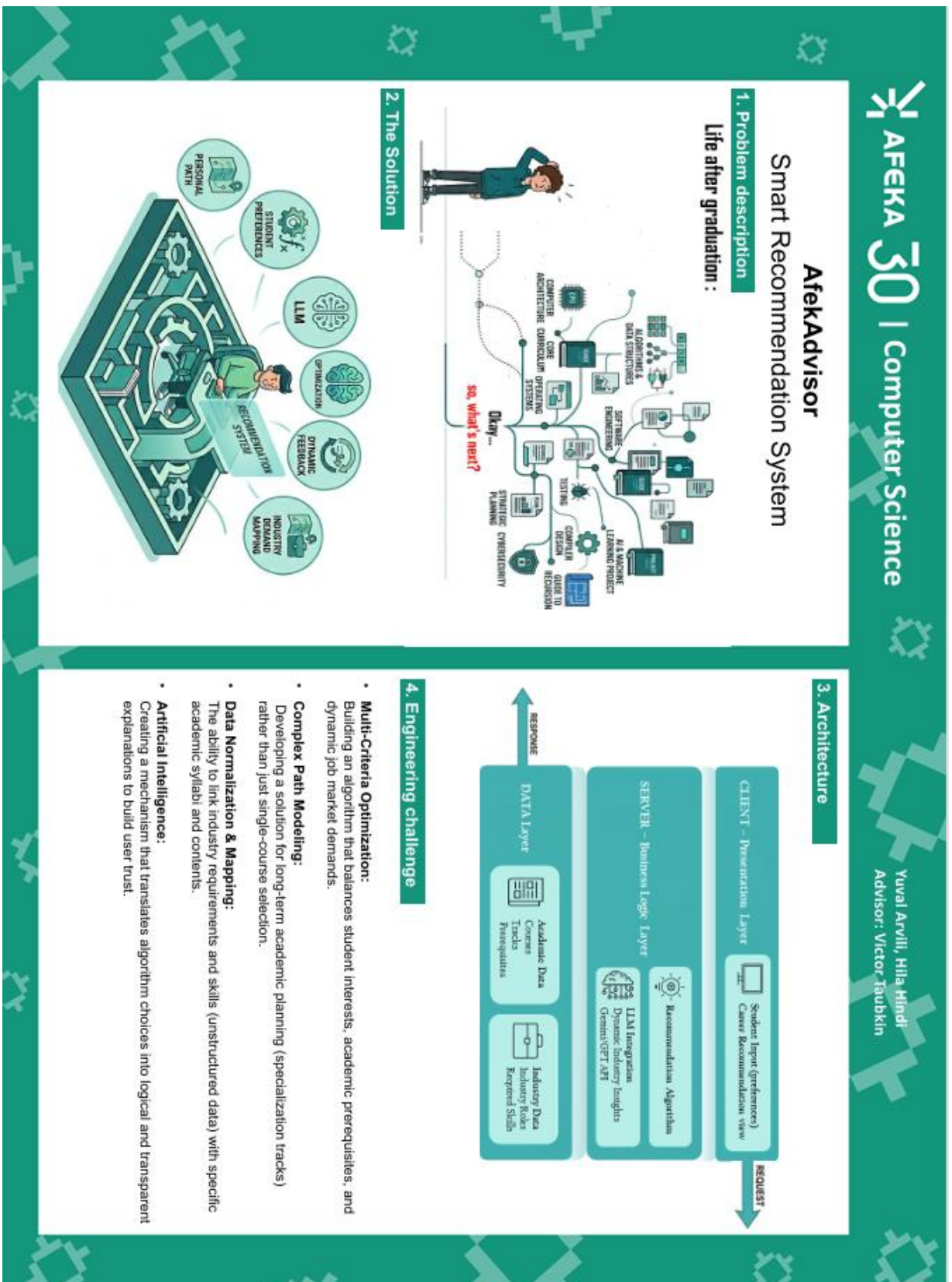
4. Adzuna Ltd. (2026). Adzuna API Documentation. Retrieved from <https://developer.adzuna.com/>
5. Docker Inc. (2026). Docker Documentation. Retrieved from <https://docs.docker.com/>
6. FastAPI. (2026). FastAPI Documentation. Retrieved from <https://fastapi.tiangolo.com/>
7. PostgreSQL Global Development Group. (2026). PostgreSQL Documentation. Retrieved from <https://www.postgresql.org/docs/>
8. React. (2026). React Documentation. Retrieved from <https://react.dev/>
9. SQLAlchemy. (2026). SQLAlchemy Documentation. Retrieved from <https://www.sqlalchemy.org/>
10. Vite. (2026). Vite Guide. Retrieved from <https://vite.dev/>

Institutional Sources:

11. Afeka, Tel Aviv Academic College of Engineering. (2026). *B.Sc. in Computer Science: Course Catalog and Final Project Guidelines*.

10. Appendices

Appendix A - Project Poster



Appendix B - Swagger API Documentation



auth

POST	/api/v1/register	Register	⌵
POST	/api/v1/login	Login	⌵

courses

GET	/api/v1/courses/	Read Courses	⌵
GET	/api/v1/courses/yearly-mandatory	Get Yearly Mandatory Courses	⌵
GET	/api/v1/courses/{course_code}/reviews	Read Course Reviews	⌵
POST	/api/v1/courses/{course_code}/reviews	Create Course Review	⌵

reviews

POST	/api/v1/reviews/submit	Submit Review	⌵
POST	/api/v1/reviews/bulk	Create Reviews Bulk	⌵
POST	/api/v1/reviews/seed-all	Seed All Reviews	⌵
DELETE	/api/v1/reviews/all	Delete All Reviews	⌵

course-pipeline

POST	/api/v1/course-pipeline/seed	Seed Courses And Reviews	⌵
POST	/api/v1/course-pipeline/extract-skills	Extract Course Skills	⌵
POST	/api/v1/course-pipeline/vectorize	Vectorize Courses	⌵
POST	/api/v1/course-pipeline/compile-ratings	Compile Course Ratings	⌵

profile

GET	/api/v1/profile/{student_id}	Read Profile	⌵
PUT	/api/v1/profile/{student_id}	Update Profile	⌵
GET	/api/v1/profile/{student_id}/history	Read History	⌵
POST	/api/v1/profile/{student_id}/history	Add History	⌵
POST	/api/v1/profile/{student_id}/history/bulk	Add History Bulk	⌵
DELETE	/api/v1/profile/{student_id}/history/{course_code}	Remove History	⌵
GET	/api/v1/profile/{student_id}/schedule	Get Schedule	⌵
POST	/api/v1/profile/{student_id}/schedule	Add Schedule	⌵
DELETE	/api/v1/profile/{student_id}/schedule/{course_code}	Remove Schedule	⌵

recommendations ^

POST /api/v1/recommendations/get Get Personalized Recommendations

GET /api/v1/recommendations/roadmap Get Roadmap

metadata ^

GET /api/v1/metadata/ Read Metadata

GET /api/v1/metadata/market-roles Read Market Roles

POST /api/v1/metadata/sync-market-roles Sync Market Roles Endpoint

jobs ^

POST /api/v1/jobs/sync Sync Jobs From Adzuna

POST /api/v1/jobs/extract-skills Extract Skills

POST /api/v1/jobs/vectorize-listings Vectorize Listings

GET /api/v1/jobs/average-vector Get Average Feature Vector

admin ^

POST /api/v1/admin/courses Create Course

PUT /api/v1/admin/courses/{course_code} Update Course

DELETE /api/v1/admin/courses/{course_code} Delete Course

GET /api/v1/admin/users List Users

PATCH /api/v1/admin/users/{user_id}/role Set User Role